The Treaty of Babel

A community standard for IF bibliography
Revision 12
16 October 2024

This document and further Babel information can be found at: babel.ifarchive.org

---

1. Preamble

1.1. Background

Since the 1980s, writers of interactive fiction ("IF") have used a variety of programs ("design systems") to create new works. Some were in-house tools used by commercial IF production houses, but from around 1990 IF has been written by a diverse Internet community, using tools written, tested and evolved within the community. The playable works of IF produced by design systems ("story files") have in almost all cases been programs for virtual machines, requiring the use of an emulator in order to play them (an "interpreter"). With few exceptions, the virtual machines used by different design systems have been mutually incompatible. Although a number of multi-format interpreters have been written, and the IF community prefers a format-independent approach to reviewing and discussing works of IF, few tools exist (as of the start of 2006) which treat all formats equally.

One obstacle to the creation of software for playing and archiving IF in a format-independent way is that few design systems, up to 2006, have compiled even basic bibliographic data into their story files – the author and title, for instance. Only one major design system has approached this problem systematically (TADS 3), though others have implemented partial solutions, or incorporate data into story files which – though not intended to be read externally – can allow the title or author's name to be found.

Most serious works of IF written since 1990 have been placed at the IF-archive, a community project which – subject to copyright issues – aims to preserve the whole corpus of IF, making it available to players of the present and future. Other web-based databases of IF exist, such as IFDB, which catalogue reviews and bibliographic data about works of IF but do not actually archive the story files: here copyright is not an issue. As a result, the IF-archive is the most complete collection of actual IF known to exist, but omits some of the most important early story files (notably Infocom's) for copyright reasons, and is not currently a useful source of bibliographic data. This is to be regretted, since such data should also be preserved. Furthermore, the IF-archive must accept story files in all formats and generate its own catalogues: the lack of any systematic way to do this imposes considerable work on the librarians.

We might say that bibliographic data for IF can be provided at three levels:

- "internally", that is, it can be printed out by the work of IF – for instance in the game banner, or in response to a typed VERSION command;
- "locally", that is, it can be read and used by the run-time software playing the story file (the "interpreter"), or by other programs which have direct access to the file;
- "externally", that is, it can exist entirely independently of the story file, for instance in a bibliographic database like the one kept by CDDB for published CDs.

All IF design systems provide internal metadata, though sometimes in a not very systematic way. TADS has provided local metadata since 2001 by means of its "Game Information" file format: a TADS story file can contain what amounts to an embedded text file listing key/value pairs. As of 2006, Inform 7 makes similar arrangements. Inform 6 is typical of other 1990s design systems in that it provides at best patchy local metadata: the Z-machine story files it compiles expose release numbers and compilation dates at certain well-defined byte offsets in the file – but not the author's name, or the title. Prior to this treaty, no design system generated metadata intended for external use.

1.2. Aims

The present treaty is an agreement between active design systems, the IF-archive and other interested parties. It provides for:

- ISBN-like unique ID numbers for story files, old and new, produced by commercial or non-commercial compilers living and dead;
- a standard format for cover art and bibliographic data;
- a web server able to provide these for a given ID number;
- a command-line tool able to identify and extract data from story files in any format;
- reference software providing a format-neutral API for reading story files, and removing "wrappers".

The aim of the treaty, and of the Babel software, is to make it much easier to write new tools for players in which the distinction of which design system created which story file is much less visible.

1.3. Signatories

The following parties are the initial signatories:

- the IF-archive;
- the design system ADRIFT;
- the design system Hugo;
- the design system Inform;
- the design system TADS;
- the Blorb wrapper format (a packaging system for story files);
- the Zoom interpreter for OS X and Unix;
- the Windows Frotz interpreter for Windows;
- the Babel software.

Current signatories now include:

- the design system Twine;
- the design system Alan.

Other design systems and tools are welcome to join. It is important to note that the treaty also provides for support of works of IF produced by design systems now obsolete or fallen into disuse.

The URL for the Treaty of Babel document and its associated resources, such as the Babel software, is:

```
https://babel.ifarchive.org/
```

Discussion of Treaty issues, including questions and proposals, may be sent to the Babel mailing list:

```
babel-if@googlegroups.com
http://groups.google.com/group/babel-if
```

2. Requirements on design systems

2.1. Provision of metadata and cover art

2.1.1. Requirements and guidelines for metadata

A design system is required to provide each newly published work with basic bibliographic information (or "metadata"). At minimum, this will be the name of the author and the title of the work, but a variety of other annotations are also possible. The treaty provides for a common core of data which can be used and shared between all tools – for instance, the author, or the date of first publication – but it does not limit the design system's ability to provide more, or to provide annotations of its own which are entirely specific to that system. In addition, the treaty makes no requirement on the design system to store the information in any particular format.

While each new work is required to have an author and a title, it is legal for the design system to set these to "Anonymous" and/or "An Interactive Fiction" if the author chose not to specify: provided that

the author has been given the opportunity to choose them, and encouraged to do so by the design system's documentation.

Requirements and guidelines for the common core of metadata are given in §5, "The iFiction format" below. To clarify the difference: requirements are such that a file of metadata (an "iFiction record") is illegal if requirements are broken, and a tool can refuse to deal with it; whereas tools must expect to have to deal with metadata which breaks the guidelines. A design system is required to pass on these guidelines to authors (rephrasing as it sees fit), and may at its own discretion enforce some of them, but it is not required to ensure that the guidelines are always kept.

2.1.2. Requirements and guidelines for cover art

A design system is not required to provide cover art for newly published works. However, if it does choose to do so, then that art must conform to the following requirements:

- Cover art must be a single square or rectangular JPEG or PNG image to be used as visual identification of the work, like a CD sleeve or a book jacket image. A PNG should not contain transparent pixels.
- Each side of the image is required to have a minimum length of 120 pixels. The aspect ratio of the image is required to be between 1:2 and 2:1 inclusive (i.e., it can be twice as tall as it is wide, or vice versa, but not more than that).

In addition, IF authors should be asked (in the documentation for the design system) to follow the following guidelines:

- Cover art for new works of IF should be square. (While non-square cover art is legal, this is provided mainly to enable scanned box covers to serve as cover art for 1980s commercial IF story files.)
- The ideal dimensions for new cover art are 960 by 960.
- Neither height nor width should exceed 1200 pixels.
- Cover art should be designed in the expectation that it will often be seen in a thumbnail form where the longer of the two sides is reduced to 120 pixels. (Thus a square image will be 120 by 120, which is about 3cm square on most screens.)
- Cover art should not actually invite prosecution for obscenity. Those writing erotica should provide reasonably coy covers, as might be acceptable to ordinary bookshops. The IF-Archive and other sites are entitled to refuse to archive any items which do not follow this guideline.

(Present builds of Inform 7 enforce the maximum-of-1200-pixels guideline as an absolute rule, but that is Inform's choice, and not a requirement of the treaty.)

2.2. The IFID unique identifier

Under the present agreement, each different published work of IF, of whatever design system (if any), shall have its own unique ID code (henceforth a "IFID"). This will be analogous to the ISBN code assigned to each different published book, except that there will be no central authority handing out numbers: instead, a design system will be allowed to generate its own IFIDs, using one of the standard unique-ID-number algorithms. It is an absolute requirement that the new ID is to be generated in such a way as to be universally unique, not just unique among projects created by this user or this design system.

The IFID shall be a sequence of between 8 and 63 characters, each of which shall be a digit, a capital letter or a hyphen.

For example, here is an Inform 7-generated IFID:

```
1974A053-7DB0-4103-93A1-767C1382C0B7
```

As with published books, where an ISBN remains the same even if the book is reprinted with corrections, the IFID should be associated with a project, *not* a specific story file compiled from it. A re-release with bug fixes should have the same IFID.

If a game is ported from one system to another (other than simply being moved from one version of a system to the next, e.g., by being moved from TADS 2 to TADS 3), the port receives a new IFID. Again, this is analogous to books: a different-format reissue of a book, such as a paperback of what was previously hardback, gets a new ISBN.

2.2.1. IFIDs for new projects

A design system should assign each different published work a unique ID code (henceforth a "IFID").

(Inform 7 creates a fresh IFID for a project whenever it opens a project which does not already have one. The IFID is generated using a UUID unique ID generating algorithm in the sense of ISO/IEC 11578:1996, with lower-case letters capitalized in accordance with the definition above. This is transparent to the user, except that a warning has been placed in the documentation: if a project already having a IFID is duplicated, and then the two copies are independently developed into quite different games, then there is a risk of releasing two different works with the same IFID. The documentation goes on to explain how to duplicate a project so that this will not occur.)

2.2.2. Game formats that embed an IFID

Modern Z-code and Glulx game files may be branded with a text such as this:

```
UUID://1974A053-7DB0-4103-93A1-767C1382C0B7//
```

(Recall that I7 uses a UUID-generating algorithm to create IFIDs.) The IFID is the string between the slashes, which will consist of digits, capital letters, and hyphens. This text is written in a character array in byte-accessible memory. Its location cannot be guaranteed, so the whole of byte-accessible memory must be scanned for the pattern `UUID://..//`.

Some Hugo story files contain an embedded UUID IFID, however the text is obfuscated by 20 being added to the value of each byte. The IFID can be located by looking for hyphens in the right pattern, though note that the hyphens are themselves obfuscated (and become "A"s).

Adrift 5 story files contain an embedded iFiction record which specify the IFID of the story file. Note that this IFID is not a valid UUID.

Alan story files may include the `UUID://...//` byte sequence. Note that the IFID may be stored using lower-case hexadecimal digits in the Alan file. These should be converted to upper case when reading the IFID.

2.2.3. The IFID for an HTML story file

A number of design systems generate output in HTML format, including Twine, ChoiceScript, Adventuron, Ink, Texture, and others.

Design systems may integrate an IFID into the output HTML by adding a `<meta>` tag to the `<head>` section of the output:

```
<meta property="ifiction:ifid" content="448E73DF-2D2F-47E7-A494-
A46B40D4CFB3">
```

(If the game comprises several HTML files, apply this to the start file.)

You should include an RDFa `prefix` attribute with the standard iFiction URI, either on the `<meta>` tag or one of its parents. This ensures that your HTML will be valid RDFa. Some examples of this

(other arrangements are possible):

```
<head>
    <meta prefix="ifiction:
http://babel.ifarchive.org/protocol/iFiction/"
        property="ifiction:ifid" content="448E73DF-2D2F-47E7-A494-
A46B40D4CFB3">
</head>

<head prefix="ifiction:
http://babel.ifarchive.org/protocol/iFiction/">
    <meta property="ifiction:ifid" content="448E73DF-2D2F-47E7-A494-
A46B40D4CFB3">
</head>
```

(Note that the `babel` tool does not check for this prefix; it just looks for the `property="ifiction:ifid"` attribute. However, other web-based metadata tools require the prefix. Consider using this validator to verify that your file validates without warnings.)

2.2.4. The IFID for other file formats

This includes executable files, game files from systems not described in this agreement, and other document formats (including plain text).

Such a file may include the IFID as a literal ASCII byte sequence, as described above:

```
UUID://1974A053-7DB0-4103-93A1-767C1382C0B7//
```

Only digits, capital letters, and hyphens are permitted between the slashes. The sequence may occur anywhere in the file, but for the sake of efficient scanning, it is better to place it near the beginning.

If the file does not contain this sequence, or if the file format precludes storing it in this form, then the IFID is the file's MD5 hash code. This will contain only hexadecimal digits, with the characters `a` to `f` written in upper case, `A` to `F`.

2.2.5. IFIDs for legacy projects

A "legacy" story file is one which pre-dates the present standard; that is, it does not incorporate an IFID in one of the ways described above.

A design system may provide an algorithm to determine a IFID for a legacy story file. The method should be reasonably straight-forward and documented here. Moreover, the design system should then provide an implementation of this algorithm in portable C: see §2.3, "The "babel" tool" below.

Legacy story files not covered by such algorithms have IFIDs equal to their MD5 hashes, as described above.

2.2.5.1. The IFID for a legacy Z-code story file

Inform 7 compiles to Z-code, but by default wraps these Z-code story files into blorbs. The end user therefore sees the result as a blorb. However, by means of cutting tools, or by turning off the default preference, it is possible for the underlying Z-code story file to get out, and therefore it is possible that interpreters or other such tools will see "loose" Z-code story files in the wild, even for I7 projects.

Such story files can be recognised because I7 brands them with a `UUID://..//` string in byte-accessible memory. Note that searching for this is unnecessary for story files pre-dating 2006.

In the absence of an IFID found branded into the story file itself, Z-code story files can nevertheless be uniquely identified using numbers found in their headers (the first 64 bytes of the files). This is preferable to using MD5 hashes since it results in human-readable IFIDs, and also because Infocom's original story files sometimes crop up with spurious tails, e.g. paddings out to what were then disc page boundaries, usually with zero bytes.

The three identifying elements of the header are:

- (a) The release number, a non-negative integer;
- (b) The serial code, almost invariably six digits stored as text;
- (c) The checksum, a 16-bit word.

Story files can be divided into pre-1990 (Infocom) and post-1990 (Inform). Post-1990 files have regular and reliable data for release number, serial code and checksum: the combination should safely identify a file. Pre-1990 files are more variable in how they handle checksums, in particular, but fortunately there are few of them.

Pre-1990 story files have IFIDs in the following form:

```
ZCODE-11-860509         Trinity, first release
ZCODE-12-860926         Trinity, second release
```

That is, "ZCODE", hyphen, release number, hyphen, serial code. In every case except three, the serial code will consist of six digits. The exceptions are:

```
ZCODE-2-AS000C          Zork I, Z-machine v1 release
ZCODE-15-UG3AU5         Zork I, Z-machine v2 release
ZCODE-7-UG3AU5          Zork II, Z-machine v2 release
```

A few other instances are known where the serial code is either non-ASCII or a numerical string which is not a compilation date in YYMMDD format. These appear to be either beta-test versions or copies modified by users to alter the serial number. In these cases, non-ASCII characters should be changed to hyphens. Some examples:

```
ZCODE-5-------          Zork I, Z-machine v1 release
ZCODE-15-------         Zork I, Z-machine v2 release
ZCODE-15-999999         Enchanter, test or modified version
ZCODE-67-000000         Sorcerer, test or modified version
```

Post-1990 story files have IFIDs in the following form:

```
ZCODE-8-040205-6630     Savoir-Faire, release 8
```

Here the format is "ZCODE", hyphen, release number, hyphen, serial code, hyphen, checksum written as four hexadecimal digits 0...9, A...F.

A sufficient algorithm to determine the IFID is therefore:

1. Look at the start of the serial code. If the serial code begins "8" or "9" or "00" to "05", go to step 3.
2. Scan byte-accessible memory for the pattern of ASCII characters `UUID://.../`, where "..." is a sequence of letters, numbers and hyphens. If this is found, that's the IFID. Otherwise continue.
3. Start with "ZCODE-", the release number, "-".
4. Copy the six characters of the serial code, converting any non-alphanumeric characters (in particular, nulls) to hyphens.
5. If the first character of the serial code is 0, 1, 2, 3, 4, 5, 6, 7, or 9 (but not 8), and the serial code is not "000000", "999999", or "------", add "-" and the checksum in hexadecimal.

The serial code characters are held in bytes 0x12 to 0x17 of a story file; the release number is an unsigned integer in bytes 0x02 and 0x03, the checksum similarly in bytes 0x1C and 0x1D. Thus, for instance, for Savoir-Faire release 8, the header bytes of the story file include –

```
0x02    0x00
0x03    0x08
0x12    0x30 = '0'
0x13    0x34 = '4'
0x14    0x30 = '0'
0x15    0x32 = '2'
0x16    0x30 = '0'
0x17    0x35 = '5'
0x1C    0x66
0x1D    0x30
```

whence "ZCODE-8-040205-6630".

2.2.5.2. The IFID for a legacy Glulx story file

Inform 7-produced Glulx files will be branded with an IFID in the same manner as Z-code files.

Older Glulx files come in two flavours: those produced by the biplatform inform compiler, and those produced by other tools.

Inform-produced glulx files provide similar information to Z-code: a release number, serial number, and checksum, which can be used to identify them.

Glulx files produced by other tools have only a checksum to identify them. In this case, the IFID is supplemented by the stated size of the initial memory map (which is equal to the size of the game file if it has not been padded).

The size of the initial memory map is stored in a Glulx file as a 4-byte value beginning at byte address 12.

The checksum is stored in as a 4 byte integer at byte address 32.

An inform-produced Glulx file can be identified by the ASCII sequence "Info" at byte address 36.

The release number is stored as a 2 byte integer at address 52.

The serial number is 6 ASCII characters beginning at address 54, traditionally the compilation date.

The algorithm for calculating the IFID of a Glulx file is as follows:

1. Scan byte-accessible memory for the pattern of ASCII characters `UUID://.../`, where "..." is a sequence of letters, numbers and hyphens. If this is found, that's the IFID. Otherwise continue.
2. The IFID begins with "GLULX-".
3. For inform-generated files, skip to step 5.
4. For non-inform-generated files, add the size of the initial memory map as eight hexadecimal digits and skip to step 7.
5. Add the release number, in decimal, followed by a hyphen.
6. Add the serial code, converting any non-alphanumeric characters to hyphens.
7. Add a hyphen, then add the checksum in hexadecimal.

2.2.5.3. The IFID for a legacy TADS2 or TADS3 story file

The IFID for a legacy ".gam" or ".t3" TADS story file is the prefix "TADS2-" or "TADS3-", followed by its MD5 hash, with hexadecimal characters a to f written in upper case, A to F.

2.2.5.4. The IFID for a legacy Hugo story file

Some Hugo story files contain an embedded UUID IFID as described above.

The IFID for a legacy Hugo story file is derived from the file header and has the following form:

```
HUGO-##-XX-XX-SERIALNO
```

The numbers are:

1. The first byte of the story file (the Hugo version number) as a decimal number.
2. The second and third bytes of the story file as 2 digit hexadecimal numbers.
3. The following 8 bytes (the serial number) with any non-alphanumerical characters converted to hyphens.

Example:

```
HUGO-25-6A-61-09-30-05
```

2.2.5.5. The IFID for a legacy Adrift story file

Adrift 5 story files contain an embedded iFiction record as described above.

The IFID for a legacy ".taf" Adrift story file is the prefix "ADRIFT-", followed by the version number as decoded from the story file, followed by a hyphen and then its MD5 hash.

Adrift files begin with an encoded text that when decoded says "Version #.##". To decode the version number you need only XOR the bytes with the constant bytes `0xA7 0x6B 0x0E 0x51`:

```
Byte  Value   Key    Result
0x08  0x94  ^ 0xA7 = '3'
0x09  0x45  ^ 0x6B = '.'
0x0A  0x36  ^ 0x0E = '8'
0x0B  0x61  ^ 0x51 = '0'
```

The IFID skips the period, so the prefix for an Adrift 3.8 story file is "ADRIFT-380-" followed by the MD5 hash.

2.2.5.6. The IFID for a legacy Magnetic Scrolls story file

The set of existing Magnetic Scrolls games is small and fixed. As it is unlikely that any new Magnetic Scrolls story files are liable to become available in the near future, we have assigned the following IFIDs for the known Magnetic Scrolls games:

```
The Pawn                MAGNETIC-1
Guild of Thieves        MAGNETIC-2
Jinxter                 MAGNETIC-3
Corruption              MAGNETIC-4
Fish!                   MAGNETIC-5
Myth                    MAGNETIC-6
Wonderland              MAGNETIC-7
```

In several cases, more than one version of a story file is publically available. In keeping with §2.2, "The IFID unique identifier", the story files use the same IFID across all versions.

Each of these can be uniquely identified by the content of bytes 12 through 32 of the story file:

```
IFID: MAGNETIC-1 00000000D5000000B40000000B00000000000000
IFID: MAGNETIC-2 000100010000000F10000000E00000000000000
IFID: MAGNETIC-2 0004000107F80000E0000000213400002070000O
IFID: MAGNETIC-3 000200010000000E0000000610000002200000l
IFID: MAGNETIC-4 00030000FF000000E0000000910000001E000001
IFID: MAGNETIC-4 000400012560000100000000710F00001D880001
IFID: MAGNETIC-5 000300010000000E00000007D0000001F000001
IFID: MAGNETIC-5 0004000124C40001000000005C5F000020980001
IFID: MAGNETIC-6 00030000DD000000600000003400000013000000
IFID: MAGNETIC-7 00040001523C0001000000004C6600002FA00001
```

In the unlikely event that a previously undocumented Magnetic Scrolls game should come to light, its IFID is "MAGNETIC-" followed by the MD5 hash of the story file.

2.2.5.7. The IFID for a legacy AGT story file

For historical reasons, AGT files are found in a variety of binary formats. The most modern of these is the AGX format used by the AGiliTy interpreter and Magx compiler. As tools to repackage older AGT binary formats into the AGX format are readily available, for the purposes of this treaty, we will assume that an "AGT story file" refers to an AGT program in the AGX format.

An AGT story file can be identified by its AGT version and a 32-bit game signature.

The AGT version number is stored as a little-endian 16 bit integer at the beginning of the game header block of of an AGX file.

The AGT version number should be one of the following:

```
value           AGT version
00000           v1.0
01800           v1.18
01900           v1.19
02000           v1.20           ("Early Classic")
03200           v1.32/COS
03500           v1.35           ("Classic")
05000           v1.5/H
05050           v1.5/F (MDT)
05070           v1.6   (PORK)
08200           v1.82
08300           v1.83
10000           ME/1.0
15000           ME/1.5
15500           ME/1.55
16000           ME/1.6
20000           Magx/0.0
```

The last digit may be replaced by '1' to indicate that the game file is a "large" or "soggy" version.

The game signature is stored as a little-endian 32-bit integer starting at the third byte in the game header block.

The location of the game header block is stored as a little-endian 32-bit integer stored at position 32 in the AGX file.

The IFID for an AGT game is constructed thus:

1. Start with "AGT-"
2. Add the AGT version as a 5-digit decimal

3. Add a hyphen, then add the game signature as 8 hexadecimal digits

2.2.5.8. The IFID for a legacy Level 9 story file

Though level9 files were designed using a platform-independent virtual machine format, they are generally encountered in a bundle with the original interpreter in the form of a disk image. In keeping with the treaty policy on dealing with archive files, the level9 module is not required to deal with compressed disk images. Users of level9 files are pointed to Paul David Doherty's l9cut program to extract these files from their archives.

The following IFIDs have been assigned for Level 9 game files. Level 9 games can be identified by their file size and checksum. For brevity, the complete table of these is not included here, but can be found in both the babel source and the source to l9cut.

```
Adrian Mole I           LEVEL9-001-n (where n is 1-9)
Adrian Mole II          LEVEL9-002-n
Adventure Quest         LEVEL9-003
Champion of the Raj     LEVEL9-004-lang (lang is an ISO-639 language
code)
Colossal Adventure      LEVEL9-005
Dungeon Adventure       LEVEL9-006
Emerald Isle            LEVEL9-007
Erik the Viking         LEVEL9-008
Gnome Ranger            LEVEL9-009-n
Ingrid's Back           LEVEL9-010-n
Knight Orc              LEVEL9-011-n
Lancelot                LEVEL9-012-n
Lords of Time           LEVEL9-013
Price of Magik          LEVEL9-014
Red Moon                LEVEL9-015
Return to Eden          LEVEL9-016
Scapeghost              LEVEL9-017
Snowball                LEVEL9-018
The Archers             LEVEL9-019
Worm In Paradise        LEVEL9-020
```

The suffix numbers -n are used to indicate story files which are parts of a sequence: several Level 9 games were split into independently playable episodes. "Champion of the Raj" is known to have had French, English and German versions.

While it is unlikely that any new binaries will come to light, several historic versions of these games are not currently available for analysis. In the event that an undocumented Level 9 game should come to light, its IFID is "LEVEL9-" followed by the MD5 hash of the story file.

2.2.5.9. The IFID for a legacy AdvSys story file

The IFID for a legacy AdvSys story file is "ADVSYS-" followed by the MD5 checksum of the file.

2.2.5.10. The IFID for a legacy Alan story file

The IFID for a legacy Alan story file is "ALAN-" followed by the MD5 checksum of the file.

2.2.5.11. The IFID for a legacy HTML story file

HTML games that lack the `<meta>` tag described above may include the text `UUID://.../` in a literal string or comment in the HTML.

Older Twine games may incorporate an IFID in a <tw-storydata> tag in the HTML:

```
<tw-storydata name="Title" creator="Twine" ifid="8665FC08-15CD-4BEC-
B15A-7F72E34F4F51" ...>
```

Otherwise, the IFID for a legacy HTML story file is "HTML-" followed by the MD5 checksum of the file.

2.2.5.12. The IFID for other legacy file formats

The concept of "executable file" is itself something of a legacy. There are many possible "executable" formats, and this document does not need to distinguish them.

For the recognized executable formats listed below, the IFID takes the form:

FORMAT-MD

Where MD is the md5 checksum of the file, and FORMAT is a descriptive identifier for the particular flavor of executable:

```
MZ      MS-DOS, Windows or OS/2 Executable
ELF     Linux ELF
JAVA    Java bytecode
AMIGA   AmigaOS executable
SCRIPT  Unix-style shell script
MACHO   MacOS X or GNU/Hurd Mach-O binary
MAC     Pre-MacOS X Macintosh binary
```

For other file formats, including text documents, the IFID is the MD5 checksum.

2.2.6. The IFID for a blorbed story file

Blorb is a wrapper format (see below) which was originally devised to enable Z-machine games to manage sound and picture resources better. It was then generalised to handle glulx story files too, and in March 2006 the specification was opened up to allow its use as a wrapper for any story file format. It should therefore be assumed that a blorb might contain almost any kind of story file.

Though blorb can package up other resources too, for our purposes a blorb archive has three ingredients:

- a story file;
- a cover image;
- a metadata chunk, containing an iFiction record.

All three are optional, but a blorb with no story file inside is not itself a work of IF and so does not have a IFID.

Any blorb generated by Inform 7 is guaranteed to contain a metadata chunk, but a small number of blorbs created between 2000 and 2005 will not have, and three blorbs in circulation (the preview games for Inform 7) use a version of iFiction (v0.90) which pre-dates the standard in this document, and is not entirely compatible with it. They will be replaced with correct versions, but may still circulate in their uncorrected form.

The IFID(s) for a blorb should be determined by the following method:

(a) If there is a metadata chunk with an iFiction record (v1.0 or later), then parse this to find the IFID(s).

(b) Otherwise, the IFID(s) for a blorb is/are the IFID(s) for the embedded story file, whatever format that has.

2.3. The "babel" tool

The babel utility is intended to be a Swiss army knife for metadata-related activities. Although the treaty agreement provides for common behaviours, it is in the babel source code that the treaty's parties collaborate directly, with functions provided by each system coming together to make up a single program. The utility is written in ANSI standard C with a view to portability: it can be compiled as a command-line tool in its own right, or can be incorporated into other tools and, in effect, provide them with an API for handling metadata and cover art independently of story file format.

A central maintainer is responsible for the whole, while each design system is responsible for its own contribution.

2.3.1. Wrappers and formats

Babel's function is to examine a (possibly wrapped) story file and read its IFID(s), metadata and cover art, and to do so in a way which is independent of the wrapper and story file formats used. As detailed below, it provides these services with a common interface regardless of format: the mechanism to extract the title and author of a TADS story file, for instance, is the same as that for a zcode story file.

By "wrapper" is meant a format which takes a story file, bundles it up with associated material, and results in a single object in the player's computer. To qualify as a wrapper format, a format should –

- (i) be used natively by one or more interpreters or browsers;
- (ii) include a story file in a format which can also exist and be played without the surrounding wrapper.

For instance, a .zip archive of a game's distribution does not count as a wrapper in this sense, since it isn't playable without unzipping: so it breaks (i). And it could not be said that the zcode format was a wrapper for the actual block of interpreted code in a story file, because the latter cannot exist separately from the header and its initial tables, which breaks (ii).

At present the only wrapper in usage is Blorb, a format used to bind up both zcode and glulx story files with bibliographic data, cover art, pictures and sounds. But we might imagine other wrappers in future: for instance, it would be possible to imagine a .rar archive which was interpreted directly, as in the example of the "comic book RAR" (.cbr) format used by collectors of digital comics. Babel is therefore written in such a way that support for other wrappers can be added as needed.

Babel is further written so that, as far as it is concerned, any supported wrapper can be used in conjunction with any supported format. Nobody has ever tried to wrap a TADS story file in a blorb, but if the blorb standard were revised to allow such a pairing, Babel would have no difficulty with the combination.

Babel examines an object file in two stages:

```
Read file   -->   Look at wrapper if any   -->   Look at format
```

It may well be that what Babel needs to find – the cover image, for instance – is provided by the wrapper. If so, it may not get to the second stage of looking at the format of the story file inside.

Code to handle wrappers is the responsibility of the maintainer of Babel. Code to handle the story file formats is the responsibility of the design system producing those formats, as outlined below.

2.3.2. Contributing to "babel"

Each design system is asked to contribute a file containing a portable, rights-free C routine to provide babel with support for its run-time format. It may be assumed that "int32" is a 32-bit or larger signed integer.

In the case of TADS 3, for instance, the file is "tads3.c": Inform contributes both "zcode.c" and "glulx.c", since it compiles to two independent virtual machines.

The specification below describes what this file should contain. Setting up such a file involves a certain amount of work: the prototype babel distribution includes a header file, "treaty_builder.h", which offers C preprocessor macros able to simplify the process. However, it is not required that these macros be used: only that the end result of writing is a file satisfying the following description.

The routine visible to, and used by, babel shall have the form:

```
int32 SYSTEM_treaty(int32 selector,
    void *story_file, int32 story_file_extent,
    void *output, int32 output_extent)
```

where the prefix "SYSTEM_" is "tads3_", "zcode_", etc., as appropriate. (When other design systems are added, these prefixes should coincide with the `<format>` value used in iFiction – see §5, "The iFiction format".) The implementation may create as many other routines, constants, etc., as it chooses, provided their names have the same prefix.

The selector may be one of the following constants, in which case the state of the pointers on entry is as given:

selector	story_file	output
GET_FORMAT_NAME_SEL	NULL	not NULL
GET_STORY_FILE_EXTENSION_SEL	NULL	not NULL
GET_HOME_PAGE_SEL	NULL	not NULL
GET_FILE_EXTENSIONS_SEL	NULL	not NULL
CLAIM_STORY_FILE_SEL	not NULL	NULL
GET_STORY_FILE_METADATA_EXTENT_SEL	not NULL	NULL
GET_STORY_FILE_COVER_EXTENT_SEL	not NULL	NULL
GET_STORY_FILE_COVER_FORMAT_SEL	not NULL	NULL
GET_STORY_FILE_IFID_SEL	not NULL	not NULL
GET_STORY_FILE_METADATA_SEL	not NULL	not NULL
GET_STORY_FILE_COVER_SEL	not NULL	not NULL

Where one of the pointers is NULL, its given extent will be 0; where it is not NULL, it must point to byte-accessible memory of the given extent. "story_file" points to a complete story file, fully loaded into memory: the routine must not write to this. This will always be the actual story file: if babel is working on a blorb which contains a zcode-format story file, say, the pointer will be to the zcode story file.

"output" points to a buffer to which text or other data can be written. On entry, the contents of this buffer are undefined. If text is written to the buffer, it should be null terminated. Under no circumstances may the buffer be written beyond its extent; but the extent is guaranteed to be at least 512 bytes. Text in the buffer must be encoded with UTF-8 Unicode.

The return value of the treaty routine will be one of the following:

```
NO_REPLY_RV
INVALID_STORY_FILE_RV
UNAVAILABLE_RV
IMPROPER_USAGE_RV
```

or else a positive integer value depending on the selector.

NO_REPLY_RV is used either if there is no meaningful return value, or if the requested data cannot be found in the story file, but there was no indication that the story file was broken or invalid. (The routine is *not* required to check the story file for validity: it is simply asked to report any problem it does hit.)

INVALID_STORY_FILE_RV means that something went wrong, and that the story file looks to be broken or invalid.

UNAVAILABLE_RV means that support for this selector is not provided, and it should be returned if the selector is not recognised (in case new selectors are added in a later revision of this standard).

IMPROPER_USAGE_RV means that the selector believes it has been called with parameters which are not as they should be, according to the specification above.

To take the selectors in turn:

```
GET_FORMAT_NAME_SEL
```

Copies the design system's format into the output buffer: e.g., "tads2". This must be the value as defined in §5, "The iFiction format" below under <format>.

Returns NO_REPLY_RV.

```
GET_STORY_FILE_EXTENSION_SEL
```

Outputs the single best extension for the given story file. Thus, ".z5" for version 5 Z-code, ".gam" for TADS 2, etc. Returns the length of the output string, or a suitable error value. This should return zero only if this particular game file should not have any extension.

```
GET_HOME_PAGE_SEL
```

Copies a URL for the design system's home page into the output buffer: e.g., "http://www.tads.org/".

Returns NO_REPLY_RV.

```
GET_FILE_EXTENSIONS_SEL
```

Copies a comma-separated list of filename extensions typically associated with the design system, flattened to lower case, into the output buffer: e.g., for zcode, ".z3,.z4,.z5,.z6,.z7,.z8" (note: ".blorb", ".zblorb", etc., are not included: babel handles blorbs automatically)

Returns NO_REPLY_RV.

```
CLAIM_STORY_FILE_SEL
```

Returns VALID_STORY_FILE_RV if this design system is certain it produced the story file, INVALID_STORY_FILE_RV if it is certain it did not, or NO_REPLY_RV if it is possible but not sure.

(The design systems usually check the file header to see if it would be valid, and only to the extent necessary for one story file format to be distinguished from the others. They do not check that the entire file is valid. For instance, the first twelve bytes of a blorb always match FORM????IFRS for some ????, and this is unlikely to be true of a randomly found file which is not a blorb. So checking those twelve bytes is sufficient to say that the file "appears to be" a blorb.)

```
GET_STORY_FILE_METADATA_EXTENT_SEL
```

Returns the extent (in bytes) of the iFiction record for the story file, plus 1 (to allow for a zero termination byte); or, if there is no metadata, returns NO_REPLY_RV. This is likely to be called

immediately before GET_STORY_FILE_METADATA_SEL, in order to establish what size of output buffer is needed to hold the metadata.

```
GET_STORY_FILE_METADATA_SEL
```

Copies the metadata for the story file, in "iFiction" format, to the output buffer, together with a zero termination byte, and returns the total number of bytes copied (including the zero termination byte); or, if there is no metadata, returns NO_REPLY_RV.

If the output buffer is too small, returns IMPROPER_USAGE_RV.

Note that it is *not* required that the story file contain metadata written in "iFiction" format: only that this selector is able to translate its metadata into "iFiction" format on request. Similarly, it is not required that the story file's own metadata be encoded in UTF-8 Unicode, only that the metadata be written in that encoding when this selector does its work. It should run quickly in comparison with file input/output, but does not especially need to be fast.

```
GET_STORY_FILE_COVER_EXTENT_SEL
```

Returns the extent (in bytes) of the cover art image in the story file, if there is one; or, if not, returns NO_REPLY_RV. This is likely to be called immediately before GET_STORY_FILE_COVER_SEL, in order to establish what size of output buffer is needed to hold the cover art.

```
GET_STORY_FILE_COVER_SEL
```

Copies the cover art for the story file into the output buffer, returning the size in bytes; or, if there is no cover art, returns NO_REPLY_RV.

If the output buffer is too small, returns IMPROPER_USAGE_RV.

```
GET_STORY_FILE_COVER_FORMAT_SEL
```

Returns JPEG_COVER_FORMAT if the cover art is in JPEG (JFIF) format; PNG_COVER_FORMAT if the cover art is in PNG format; NO_REPLY_RV if there is no cover art. No other formats of cover art are legal.

```
GET_STORY_FILE_IFID_SEL
```

Finds the IFID(s) and copies them as a comma-separated list into the output buffer, and then returns the number of IFIDs in the list; or returns NO_REPLY_RV if no IFID could be determined.

2.3.3. Algorithm for determining the format of a story file

(a) If a story file is a blorb, then the blorb executable chunk records the format of the real story file inside. This should be checked by seeing if the relevant format's CLAIM_STORY_FILE_SEL selector is happy: if not, then babel is allowed to declare the story file invalid. The blorb is similarly deemed invalid if this format disagrees with the format claimed in its metadata chunk (if there is one).

(b) If not, then we have a story file which might be one of many formats, or might even be (say) an MS-DOS executable. The filename extension is considered as providing a clue, so:

- (i) Babel runs through the formats, polling each with GET_FILE_EXTENSIONS_SEL to see if the file extension is one that any of them know. This makes a list of "likely" formats (extension matching) and a list of the remaining "unlikely" ones.
- (ii) Babel offers the story file to each likely format in turn with CLAIM_STORY_FILE_SEL; then to each unlikely format in turn. The first to claim the story file is the winner. If no format claims it, then the format is "unknown".

- (iii) Within each list – the likely and unlikely lists, that is – the formats are checked in popularity order, i.e., in order of the size of the published corpus for the formats: so probably zcode, then tads2, then any others which may provide treaty routines.

3. Guidelines for interpreters and browsers

An "interpreter" is a program which plays the story file: for some design systems a single "run-time" program is provided; for others a variety of rival interpreters exist. A "browser" is a program which manages and offers a choice of story files. One may imagine an interpreter which is not a browser – it would only play a single story file; and one may imagine a browser which is not an interpreter – it would play story files by calling an external application (i.e., an interpreter for the given format). But it is also possible to write tools which are both browsers and interpreters.

There are no formal requirements on these tools, but the spirit of this standard is that the following will be considered good practice. Some of these guidelines are applicable to interpreters, some to browsers, and some to both.

(a) To use basic bibliographic data if it is available, even if only to give windows and/or saved games sensible titles, or to provide an "About this Game" menu option;

(b) If possible, to be able to show a preview of a game based on its cover art and bibliographic data, before play begins;

(c) If possible, to be able to browse through a selection of games, tabulated using their bibliographic data;

(d) If possible, to be able to show previews and to browse story files compiled by any design system signing up to the standard, though probably "playing" them by launching an external application – the run-time for the design system in question; failing that, to display a message such as "Lady Eustace's Diamonds appears to be a work of interactive fiction in XYZ format. You will need to obtain special XYZ software in order to play it." and a link to the XYZ home page.

(e) If possible, to allow the user to edit the metadata of games in their collection (in a way which does *not* involve alteration to the story file, but simply editing of the interpreter's own copy of the metadata for its archive of games – see below);

(f) If possible, given an object with no metadata, or no obvious file type, to attempt to fetch metadata and/or cover art for it from the IF-Archive (see below), and then employ this in the ways suggested above.

Interpreters or browsers fetching cover art from the IF-Archive are *required* to cache the images they fetch, in order to minimise bandwidth usage. They should be written on the assumption that a fetch may take several seconds.

4. Requirements on the IF-archive

When authors complete and wish to publish a new work, they "upload" it to the IF-archive's incoming folder. There it remains until one of the librarians works through the new uploads and files them appropriately: we shall call this "accessioning". Users of the IF-archive can browse the directories freely, and download as they like.

As part of this new standard, however, the IF-archive will offer further services: it will provide metadata on request ("serving") to any interpreters or other tools which ask for it, and it will accept metadata on existing works (whether or not archived) as they are provided by volunteers ("contributing").

Note that other archives could provide similar facilities: it would be natural to be able to access IFDB via the IFID or TUID, and this would make a neat way for interpreters to point to reviews, etc., on a

game.

4.1. Uploading

A story file may be uploaded as a single object, such as "Henrietta.zblorb" or "DDD.gam", or as a zipped archive such as "Bronze.zip", which contains both the story file and its associated external resources – manuals, maps, etc., typically in plain text or PDF format, and might expand to a short set of files as follows:

```
Bronze.zblorb
Bronze Manual.pdf
Map.pdf
solution.txt
```

4.2. Accessioning

(a) When a newly uploaded work is accessioned, the librarian should run the "babel" command-line tool to examine the story file. This will produce the IFID (always), file(s) containing the iFiction record (if any) and the cover image (if any).

If the iFiction record contains several IFIDs, as for instance to list multiple releases of an Infocom game under a common IFID (see §5.5, "Identification" below), "babel" will create one copy of both the record and the image, filenamed with each IFID.

(b) If any IFID duplicates one already held by the IF-archive, the librarian will investigate to see if they are essentially different works:

- (i) If so, the new upload will be rejected, and the author will have to resubmit. Such accidents are likely to be rare.
- (ii) If not, the upload will be allowed, and will become the "main" version held by the IF-archive under that IFID: the new upload's cover art and iFiction will replace the old. The librarian may, at their discretion, delete the old story file, so that only the new version remains. Such replacements with revised versions are likely to be fairly common.

(c) If there is an iFiction file, it will be filed in some appropriate directory as

```
XXXXX.iFiction
```

where XXXXX is the IFID.

(d) If there is a cover image, it will be reduced in size to the standard thumbnail dimensions: that is, such that its longer side is 120 pixels. (This will usually mean it is 120 by 120.) A standard Unix tool such as "sips" should be able to do this easily enough. The result will be filed in some appropriate directory as

```
XXXXX.png  or  XXXXX.jpg
```

where XXXXX is the IFID.

4.3. Serving

Note: The IF-archive has not implemented this section. An effort is in progress to support metadata in a manner compatible with IFDB. This may result in changes to the plan proposed here.

The IF-archive will arrange that page hits are URLs in the following IFID-based form result in predictable data served back:

```
http://babel.ifarchive.org/metadata/IFID
```

- fetches the iFiction record for this IFID, giving it the mime type of a UTF-8 Unicode text file, as indeed it is; or fetches a 404 error page if no metadata exists for this IFID

```
http://babel.ifarchive.org/cover/IFID
```

- fetches the cover art, as a JPEG or PNG, or a 404 error page

```
http://babel.ifarchive.org/download/IFID
```

- fetches the story file, or a 404 error page

```
http://babel.ifarchive.org/holdings/IFID/*
```

- fetches the file * associated with this IFID. Here, "*" would be the leafname: "Bronze Manual.pdf", for instance

4.4. Contributing

Note: The IF-archive has not implemented this section. An effort is in progress to support metadata in a manner compatible with IFDB. This may result in changes to the plan proposed here.

(a) The IF-archive will provide mechanisms for volunteers to upload records on games which are not themselves being uploaded, via a web page form and/or an ftp folder for uploads.

Volunteers will be asked to upload one record for each game, in the form of a folder (or zip archive) which contains the following:

```
metadata.iFiction
cover.jpg   or   cover.png (optional)
url.txt (optional)
```

The archive is free to reject uploads not following this rule.

The "url.txt" file contains the location in the Archive, if data is being provided on a story file which is already there. Volunteers will be instructed to supply a complete URL.

(b) Such uploads will also be periodically accessioned by a librarian. Each .iFiction file will be run through "babel", which will reject it if it does not properly conform to the iFiction format, but will otherwise print out the IFID(s) and make canonically-named copies, one for each IFID. The image will also be canonically named, reduced if necessary as in 4.2 above, and duplicated if necessary.

- (i) If, in the librarian's opinion, a better record already exists for that IFID, the new one will be rejected.
- (ii) If a "url.txt" file is provided, the librarian will check the IFID of the story file at that location using "babel": if it differs from the one claimed, the IFID produced by "babel" is deemed to be the correct one.

(c) Image and .iFiction file(s) will then be stored, ready to be served as in 4.3 above.

5. The iFiction format

5.1. Purpose and scope

This format was created for Andrew Hunter's Z-machine interpreter "Zoom", which collects and organises interactive fiction story files in the same way that Apple's "iTunes" collects and organises music files. "iFiction" was the format of Zoom's database of metadata tags for its story files: compare, for instance, the XML file output by the "Export Library" menu option in iTunes. The format was used

experimentally by Zoom, then amended and introduced as a new segment into the Inform wrapper format Blorb: the first published story files to contain embedded iFiction records appeared in February 2006, though this treaty revises the format substantially from that original state.

The metadata associated with a story file will be called its "iFiction record". If we compare the story file to the pages of a book written by the author, then the iFiction record is the text printed on the cover, title and imprint pages – the part of a book normally put together by the publisher. The content is comparable with the release information handed out to distributors by record companies, or the Neilson book database which feeds Amazon and similar sites.

The iFiction record is primarily intended for bibliographic purposes: that is, purposes such as identification, classification, automated downloading, librarianship. It is not suitable for indexing the textual content of a work of IF, nor for expressing machine-followable cross-references between works of IF, or between IF story files and otherwise unrelated cultural artefacts (such as printed books). While the description of work A could certainly say that is a translation into French of work B, for instance, or an adaptation of Dickens's "A Tale of Two Cities", iFiction does not provide machine-followable links. (Equipped with Google, any reader of iFiction data would have no trouble in finding such remote resources manually.) iFiction records are written in XML rather than any more sophisticated data representation language so that only very light tools are needed to handle it: iFiction text is easy to read, write and transmit via email or the Web.

The iFiction record is also not intended to specify a license for the story file, though design systems are entitled to record this in their own segments of the iFiction format if they wish to do so. Nothing should be inferred from the absence of such license information in the iFiction record. Similarly, the copyright record for a story file is recorded in the file itself, and not in the iFiction record.

5.2. Encoding

An iFiction record is encoded in UTF-8 Unicode.

Where iFiction data is stored in a file, this file should have the file extension ".iFiction". It is permitted to begin with the UTF-8 encoding of the Unicode BOM sequence, 0xEF 0xBB 0xBF, which serves as an optional marker of the encoding type; but it is not required to do so.

5.3. Storage in files

An iFiction file should take the form:

```
<?xml version="1.0" encoding="UTF-8"?>
<ifindex version="1.0"
xmlns="http://babel.ifarchive.org/protocol/iFiction/">
    <!-- Bibliographic data recorded by Inform 7 build 3F47 -->
    ...
</ifindex>
```

where "..." is a sequence of one or more story file records. The comment is optional and should not be parsed by tools looking at the record; Inform writes in such a comment merely for the sake of helping to debug if problems are found in the records it writes.

(The ability to hold more than one story file record is provided so that an interpreter, or similar tool, can use the same schema for a database of all the metadata known to it.)

The name-space is defined in the root element `<ifindex>` to facilitate future developments, and to make it easier for XML tools to merge data from iFiction files.

5.4. Story file records

Data on an individual story file is given within <story> ... </story> tags. This is in turn subdivided:

```
<story>
    <identification>
        ...
    </identification>
    <bibliographic>
        ...
    </bibliographic>
    ...
</story>
```

We shall call these subdivisions "sections". The sections are:

tag		who originates the data
<identification>	mandatory	design system
<bibliographic>	mandatory	author
<resources>	optional	author
<contacts>	optional	author
<cover>	optional	design system
<releases>	optional	design system
<colophon>	optional	design system, or browser tool, etc.
<annotation>	optional	third parties, player's tools, etc.
<zcode>	optional*	design system
<tads2>	optional*	design system
<tads3>	optional*	design system
<glulx>	optional*	design system
<hugo>	optional*	design system
<adrift>	optional*	design system

* Permitted only if the project belongs to this format, so that at most one of these can be given: but it is legal not to give it at all.

<annotation> is a special case. A design system is forbidden to produce an <annotation> section in the iFiction records it produces. The section is expressly intended for third parties: for players, annotating their own collections using iTunes-like browsers; for review sites such as IFDB, recording reviews, ratings and other user experiences; and so on. Whether the IF-archive will serve records containing <annotation> is a matter of policy for the archive, and not governed here.

All other sections can be produced by a design system, but those marked above as originating with the author will stem fairly directly from the author's explicit instructions, whereas the others are likely to be generated automatically on their behalf. (For instance, the author will at some stage actually type the title, but the design system would likely calculate the pixel dimensions of the cover art, the serial number of the compiled story file, etc., automatically on each compilation.)

5.5. Identification

The <identification> section is mandatory. The content here identifies to which story file the metadata belongs. (This is necessary because the metadata may be held on some remote server, quite separate from the story file.)

This section contains two mandatory tags and one optional one, as in the following example:

```
<identification>
    <ifid>1974A053-7DB0-4103-93A1-767C1382C0B7</ifid>
```

```
    <format>zcode</format>
    <tuid>plvzam05bmz3enh8</tuid>
</identification>
```

5.5.1. <ifid>

Note that there *must* be at least one IFID: this may, of course, be an MD5 checksum. With all new IF projects, a design system should supply one and only one IFID. Subsequent releases with bugs fixed, etc., will have the same IFID as the first-published story file.

But in an iFiction record for an old commercial work of IF existing in multiple releases, it is legal to quote more than one IFID. For instance, here is the <identification> section of a record which the IF-archive could usefully hold on Infocom's "Sorcerer" (1984):

```
<identification>
    <ifid>ZCODE-18-860904</ifid>
    <ifid>ZCODE-15-851108</ifid>
    <ifid>ZCODE-13-851021</ifid>
    <ifid>ZCODE-6-840508</ifid>
    <ifid>ZCODE-4-840131</ifid>
    <ifid>ZCODE-67-000000</ifid>
    <format>zcode</format>
</identification>
```

When an iFiction record relates to a second or subsequent edition of a work which previously had a different IFID, it is similarly legal to give the IFIDs of earlier versions. For example, for Emily Short's "Savoir-Faire" (2004),

```
<identification>
    <ifid>731F0480-5CB5-4340-B65B-384FC6B1F5B4</ifid>
    <ifid>ZCODE-8-040205-6630</ifid>
    <format>zcode</format>
</identification>
```

This is generated by Inform 7 when it makes a new blorbed edition out of an old Inform 6-compiled story file: the I7 IFID comes first.

In all cases of doubt, if a single IFID must be extracted, the first one occurring in the story file should be used. Note that the first IFID in the example for "Sorcerer", above, is the final release – the best one to have.

5.5.2. <format>

The <format> is also mandatory, and here there can be only one value supplied. It is the format of the story file to be executed, not the name of the design system which compiled it; and in the case of a wrapper format such as Blorb, it is the format of the story file inside, so that "<format>blorb</format>" is incorrect. The value of <format> may therefore be one of the following:

```
zcode, glulx, tads2, tads3, hugo, alan, adrift, level9, agt,
magscrolls, advsys, html, executable
```

No distinction is made here between sub-versions (e.g. v4 of the Z-machine vs. v8 of the Z-machine): the distinction between TADS 2 and TADS 3 is made because their virtual machines are so completely different that there is no joint interpreter. (It is thought that no TADS 1 files exist in the wild: if any are found, they have format "tads2" for purposes of this Treaty.) The idea is that the main interpreters for any given format are always likely to be able to handle all the likely sub-versions of that format which

they encounter. A design system wanting to record more detail on the format is free to do so within its own part of the iFiction record.

The format "html" means that this is an HTML file, perhaps collected with associated files such as CSS and Javascript. The intended interpreter is a standard web browser.

The format "executable" means that this is not a virtual-machine-based story file but a program for a physical machine: say, an old MS-DOS executable. There is therefore no applicable interpreter. This format is only likely to be seen in external metadata, at sites such as the IF-archive.

Other format values may be added in future revisions. A value not listed above may be assumed to be in informal use by the users of an IF system not yet part of the Treaty. We expect that such systems will be added to this document in due course.

(New format values must not collide with tag names already defined by this document. This ensures that format-specific tags can be used without confusion.)

5.5.3. `<tuid>`

This is an optional tag, whose value is a string of letters and digits. If supplied, it should be the Interactive Fiction Database identifier (TUID) for the work: for instance, "Curses" is "plvzam05bmz3enh8".

The Interactive Fiction Database is found at ifdb.org.

5.5.4. `<bafn>`

This is an optional tag, whose value is a non-negative integer. If supplied, it should be the "Baf's Guide to the IF Archive" identifying number for the work: for instance, "Curses" is 55; "The PK Girl" is 1897.

This tag was provided solely for the convenience of people who wish to translate existing metadata at Baf's Guide to new iFiction records, particularly for items in obscure or long-dead formats. As Baf's Guide is no longer extant, it is not expected that iFiction records generated for new games will contain this tag.

5.6. Bibliographic

The `<bibliographic>` section is mandatory.

This section contains bibliographic data. The tags inside can occur in any order and all are optional except `<title>` and `<author>`, which are mandatory. Example:

```
<bibliographic>
    <title>Lakeside Living</title>
    <author>Emily Short</author>
    <language>en-US</language>
    <headline>An Interactive Example</headline>
    <firstpublished>2006</firstpublished>
    <genre>Fiction</genre>
    <group>Inform</group>
    <forgiveness>Merciful</forgiveness>
    <description>This is example 194, for what it's worth.
</description>
</bibliographic>
```

The rules below are once again divided into requirements (no system generating iFiction records is permitted to violate these) and guidelines (which authors should be encouraged to follow, and design

systems are at liberty to force them to follow, but which other tools should not assume have always been followed).

General requirements for the <bibliographic> section:

- These values are all textual except <seriesnumber>, which is a non-negative integer.
- A textual value is a sequence of one or more Unicode characters in which the following string escapes, and no others, must be used:

```
&       &amp;
<       &lt;
>       &gt;
```

- A textual tag value other than <description> must not contain any white space except for single spaces, which must not occur at the start or end of the string. (Thus tab characters, non-breaking and thin spaces, etc., are prohibited, as is leading or trailing space, and doubled spacing such as "... here. Between sentences...".)
- A textual tag value is not permitted to be the empty string.
- Uniquely, the <description> value is permitted to contain paragraph breaks, which must be encoded as "
". The description is nevertheless written in plain text, not HTML, and can contain no other tags. The other textual values must not contain any tags in angle-brackets.
- The <description> is permitted to contain white space in the form of newlines and tab characters, and to contain multiple white space characters in succession: these are always treated as single spaces. However, the <description> must not begin or end with white space, and must not be empty.

General guidelines for the <bibliographic> section:

- While there is in principle no upper limit on the length of the textual values, writers of interpreters and browsers will wish to truncate any over-long values in their own displays: it is recommended that such truncation occur at 240 characters for each tag except <description>, where up to 2400 characters should be permitted. Authors should be made aware of this likely truncation.

5.6.1. <title>

The <title> tag is mandatory. If no title is known, the correct form is

```
<title>An Interactive Fiction</title>
```

Guidelines:

- A design system should make determined efforts to get the author to provide a title.
- <title> text should be given in normal title capitalisation for the work's language: for English, that would mean caps on the first word, and on each subsequent word other than prepositions ("at", "between", etc.), articles ("the", "a", "an") and connectives ("and", "of", etc.).
- If there is a subtitle, best form is to use a colon in between title and subtitle:

```
<title>Zork II: The Wizard of Frobozz</title>
```

- Interpreters and other tools which search or sort on this field should do so case-insensitively, and following normal natural language conventions for the language in question (which may be assumed to be English if there is no other indication). For English, that would mean disregarding any of the initial articles "The", "A", "An". Thus in an alphabetical listing by title, "A Change in the Weather" should appear under C.

5.6.2. <author>

The <author> tag is mandatory. If no author is known, the correct form is

```
<author>Anonymous</author>
```

Guidelines:

- A design system should make determined efforts to get the author to give their name, or else take some active step to declare themself anonymous.
- The `<author>` must be a name or names, and must not be a sentence or partial sentence describing the authorship (e.g. "by James Madison", "written by Geoffrey Chaucer", "a novelty by Jack Djill").
- The `<author>` tag can certainly contain pseudonyms, but should not be used for e.g. email addresses or IRC handles.
- Names should be capitalised as the author wishes, except that they should not be given entirely in upper case: for instance "Humphrey Sponge", "Donna de Vries", "Marmaduke X. ffrench". Initials should be followed by full stops, as shown. If there are multiple initials, these should be spaced: "T. F. Shuttlecock", not "T.F. Shuttlecock".
- The author of a commercially published work should be its actual author, not the name of the company. Thus the `<author>` of "Spellbreaker" is "P. David Lebling", not "Infocom".
- Where there are co-authors, a list can be given. Use "and", not an ampersand. Thus: "Emily Short and Andrew Plotkin". If there are three or more authors, the serial comma may be used or not, as the authors prefer; and their names can occur in whatever order they prefer.
- Interpreters and other tools which search or sort on this field should, again, do so case-insensitively.

5.6.3. `<language>`

The `<language>` tag is optional, and records the language in which the work's text is written (or primarily written, if the work uses multiple languages). This information can help potential users identify works written in languages they know, and could also be used as a hint to text-to-speech converters or other natural language analysis tools.

The value is required to be an ISO-639 two- or three-letter language code, optionally followed by a hyphen and an ISO-3166 country code. (For English, typical specifiers would be en-US, for US English, or en-GB, for British English.)

Guidelines:

- Design systems may assume that the language is English, but should provide the author with the opportunity to change this if it isn't.
- Where the author has not specified any dialect, none should be given: thus the default should be "en", not "en-US" or "en-UK".

5.6.4. `<headline>`

The `<headline>` tag is optional. This is the quasi-subtitle traditionally used in games which follow the Infocom style: for instance,

```
<headline>An Interactive Mystery</headline>
```

Guidelines:

- A suitable default value is "An Interactive Fiction".
- This value should be cased as for `<title>`.

5.6.5. `<firstpublished>`

The `<firstpublished>` tag is optional. This is the date of first publication: it may well not be the year in which the most recent story file was compiled. (For example, `<firstpublished>` for Zork I might be 1980, though its final compiled release was in November 1987.)

It is required to have either the form YYYY or YYYY-MM-DD, giving either a year or an exact day. (YYYY-MM is illegal, and the year must be given as four digits.)

Guideline:

- Publication implies availability to the public, whether as a free download or for sale: availability to groups of beta testers does not count.

5.6.6. <genre>

The <genre> tag is optional. This is the author's choice of description of the genre of their work.

Guidelines:

- This is always going to be a highly subjective tag, and one which no overarching consistency can reasonably be expected. The author is encouraged (though not required) to use one (only) of the following:

```
    Children's Fiction, Collegiate Fiction, Comedy, Erotica, Fairy
Tale,
    Fantasy, Fiction, Historical, Horror, Mystery, Non-Fiction, Other,
    Religious Fiction, Romance, Science Fiction, Surreal, Western
```

- These categories are loosely based on those currently used by bookshops, as amended by comparison with Baf's Guide.
- "Fiction" is intended for works whose essential purpose is literary, in a way which trumps any subject they happen to have: if Julian Barnes writes a mystery, for instance, a bookshop will shelve it with modern novels rather than in the detective stories section, whereas P. D. James's Adam Dalgliesh mysteries will end up filed with detective fiction even though she has appreciable claims to be an important novelist.
- "Comedy" is used rather than "humour"/"humor" to avoid the ambiguous spelling. This genre includes parodies.
- "Non-Fiction" would be used for a work of IF which is essentially a presentation, perhaps in a novel interactive format, of true information. A meticulous simulation of the Great Exhibition of 1851, for instance, might qualify.
- The distinction between "Surreal" and "Other" is that "Surreal" works contain at least some semblance of narrative, whereas "Other" is intended for works which "abuse" the format to present some entirely different sort of game – Tetris, say, or Minesweeper.
- A suitable default value is "Fiction", but there is no need to give this tag at all.

5.6.7. <group>

The <group> tag is optional. This places a given work of IF as belonging to a given group of works.

Guidelines:

- This is *not* the name of a series. If the work belongs to a series (whether numbered or not), that should be recorded in <series>.
- The <group> for every Infocom story file is "Infocom", and similarly for the other pre-1990 companies.
- This tag is primarily provided so that iTunes-like browsers can usefully subdivide collections. As such, it is likely to be edited on many people's collections: the end user might choose to set the <group> for a selection of files to "IF Competition 2002", or "Mystery House Art Exhibition", etc.
- It belongs in the <bibliographic> section partly so that the design house can be recorded for old commercial works (see above), and partly because the author (or design system) might as well offer at least a suggestion. Inform 7 sets this tag to "Inform".

5.6.8. <description>

The <description> tag is optional, and should contain the author's outline of the work.

Uniquely, this tag may contain paragraph breaks, which are written "
". No other HTML-like tags are permitted: this is plain text. Multiple spaces, tabs, newlines, etc., are treated as single spaces (thus typing a skipped line does not achieve a paragraph break).

Guidelines:

- The value here should be thought of as analogous to the back cover blurb on a book, which might be read by someone picking the book up in a casual way. Like a cover blurb, it should feel able to "sell the product" as well as merely to itemise it.
- Quite a good default is to use the initially-printed paragraph of the story, then a supplementary paragraph explaining better, or talking about the author.

For instance:

```
<description>Sharp words between the superpowers. Tanks in
East Berlin. And now, reports the BBC, rumors of a satellite
blackout. It's enough to spoil your continental
breakfast.<br/>But the world will have to wait. This is the
last day of your $599 London Getaway Package, and you're
determined to soak up as much of that authentic English
ambience as you can. So you've left the tour bus behind,
ditched the camera and escaped to Hyde Park for a
contemplative stroll through the Kensington
Gardens...<br/>Trinity, written by Brian Moriarty and
first published by Infocom in 1986, is an intriguing fantasy
rooted both in traditional adventure games and in historical
research on the development of the atomic bomb. Dark,
literary and sophisticated, Trinity is one of the most
influential works of IF ever written.</description>
```

5.6.9. <series> and <seriesnumber>

Both of these tags are optional. However, if <seriesnumber> is given, then it is required that <series> is also given. (The reverse is not the case: an unnumbered series is permitted.) <series> holds the name of the series, and <seriesnumber> the position in the series, which must be a non-negative integer. (It must *not* be text such as "III" or "Fifth Part".)

Guidelines:

- This should be used when the game is intentionally part of a trilogy, or other set of works intended to be played in conjunction with each other.

For instance, the iFiction record for "Spellbreaker" might contain:

```
<series>The Enchanter Trilogy</series>
<seriesnumber>3</seriesnumber>
```

5.6.10. <forgiveness>

The <forgiveness> tag is optional. The value is information (normally from the author) on how forgiving the story is, on the Zarfian scale. The value is required to be one of the following, and in this casing (i.e. initial letter the only one in upper case):

```
Merciful    - cannot get stuck
Polite      - can get stuck or die, but it's immediately obvious that
            - you're stuck or dead
```

```
Tough       - can get stuck, but it's immediately obvious that you're
            about to do something irrevocable
Nasty       - can get stuck, but when you do something irrevocable,
            it's clear
Cruel       - can get stuck by doing something which isn't obviously
            irrevocable (even after the act)
```

This scale is understood to be less clear-cut and less generally applicable than was believed when this Treaty was first written. It is left to the author to decide what value is appropriate, or whether any value is appropriate.

5.7. Resources

The `<resources>` tag is optional. This section, if present, details the other files (if any) which are intended to accompany the story file, and to be available to any player. By "other" is meant files which are not embedded in the story file. (So, for instance, pictures in a blorbed Z-machine story file do not count as "other".)

It contains one or more `<auxiliary>` blocks. For instance:

```
<resources>
    <auxiliary>
        <leafname>Bronze Manual.pdf</leafname>
        <description>Manual</description>
    </auxiliary>
</resources>
```

5.7.1. `<auxiliary>`

This tag records the details of a single external resource. It contains two compulsory tags:

The `<leafname>` is the filename, with any directory path removed. By convention all resources should be filenamed with standard file extensions, as might be used on Windows or Mac OS X. The leafname should not include full stops (except for the extension), slashes of either persuasion \ or /, or colons.

If the leafname has no extension, then it is the name of a folder. This must contain a small web-site, with only internal links, and whose home page is "index.html" inside the folder.

The `<description>` is a brief textual description of what the resource is, such as might be used in a menu of downloads.

5.8. Contacts

The `<contacts>` tag is optional.

If present, it can give one or both of a home page and an author's contact email address. These should only be given where it seems likely that they will be valid for an indefinite period of time.

```
<contacts>
    <url>http://www.inform-fiction.org/</url>
    <authoremail>graham@gnelson.demon.co.uk</authoremail>
</contacts>
```

5.8.1. `<url>`

The `<url>` tag is optional. It must be a valid, absolute URL, and the protocol must be "http://" or "https://".

5.8.2. <authoremail>

The <authoremail> tag is optional.

Email addresses should be given "raw", without explanation of their owners: so, "Graham Nelson graham@gnelson.demon.co.uk" is not legal (and would cause trouble because of needing to escape the angle brackets).

Multiple authors, or multiple email addresses, can be listed by separating the entries with commas.

5.9. Cover art

The <cover> tag is optional, except that it is mandatory for an iFiction record embedded in a story file which contains a cover image; and the information must, of course, be correct.

If given, this section contains the tags below. The <format>, <height>, and <width> tags are required. The <description> tag is optional but recommended.

```
<cover>
    <format>jpg</format>
    <height>960</height>
    <width>960</width>
    <description>A man wearing an unusual hat.</description>
</cover>
```

5.9.1. <format>

This is required to be either "jpg" or "png". No other casings, spellings or image formats are permitted.

5.9.2. <height> and <width>

In pixels: these are positive integers.

5.9.3. <description>

This is a brief textual description of the image. An interpreter might display this as an alternative when graphical display is not available, or when supporting nonvisual users. (Tag added in revision 8.)

5.10. The format-specific tags

An iFiction record can, optionally, have one of the following:

```
<zcode>, <glulx>, <tads2>, <tads3>, <hugo>, <alan>, <adrift>,
<level9>, <agt>, <magscrolls>, <advsys>, <html>,
<executable>
```

It may only have the tag which matches the <format> value in the <identification> section.

Each design system may specify whatever schema it likes for keys and values in its own section. The understanding is that tools not associated with that design system will probably not use this data. In other words, only TADS-based utilities or interpreters are likely ever to look at the contents of <tads3>, etc.

The specification for any given format can be revised without the need to ask other parties: it "belongs" to the design system which generate the format. (Subject to not doing anything which would make iFiction files impracticable for others, such as including uuencoded binary data, or violating XML rules.)

5.10.1. <zcode>

This section contains only optional tags.

```
<zcode>
    <version>8</version>
    <release>7</release>
    <serial>930723</serial>
    <checksum>FF3D</checksum>
    <compiler>Inform 7 build 3G08</compiler>
    <coverpicture>1</coverpicture>
</zcode>
```

The values of these tags are meaningful only when the iFiction record is embedded with the story file, not when the iFiction record exists externally, since they detail a specific compilation, not the project in general. The first four should match the header entries in the story file most recently compiled; the fifth is the most recent compiler used. Version is the Z-machine version.

(Inform 7 generates `<checksum>` only when blorbing up an existing I6 story file in new covers. In such cases, it generates the `<compiler>` by looking to see whether earlier Informs or Infocom's ZIL compiled the story file.)

5.10.1.1. `<coverpicture>`

`<coverpicture>` is used for a Blorbed z-code story file which contains cover art: it's the number of the picture which is used as the cover.

5.10.2. `<tads2>` and `<tads3>`

TADS 2 and TADS 3 story files are entirely different in format, and for purposes of the Treaty are considered independent. However, their format-specific sections – `<tads2>` and `<tads3>` – have the same contents.

This section contains only optional tags.

```
<tads2>
    <version>7</version>
    <releasedate>1993-07-23</releasedate>
    <presentationprofile>Plain Text</presentationprofile>
    <htmldescription>This is a <b>terrific</b> game.</htmldescription>
</tads2>
```

5.10.2.1. `<presentationprofile>`

The name of the recommended "presentation profile" for the game. This is a hint that gives the run-time interpreter an idea of the style of the game's user interface; interpreters can use this information to choose the most appropriate display settings, such as fonts and colors. Interpreters need not use this information at all; it's purely advisory. This value can be a user-defined profile name, or one of these pre-defined values:

Plain Text indicates that the game is entirely text, with no graphics and with text formatting limited to "highlighted" text (i.e., the traditional TADS 2 highlighting, which is usually rendered as bold-face or equivalent). Multimedia indicates that the game makes use of HTML text formatting effects (such as fonts, text colors, text sizes, italics, and tables) and/or displays graphics. Default isn't a true profile, but rather explicitly selects the default profile. Some interpreters let the user choose a particular profile as the default, in which case this will select that profile. Authors can, if they wish, specify custom profile names of their own creation here, but authors doing so are advised that (1) interpreters will not generally be able to infer anything from profile names other than ones defined here, and (2) other standard profile

names may be added here in the future, so custom names that authors choose could conceivably collide with future additions.

The profile name isn't sensitive to case (that is, "multimedia" is treated as equivalent to "MULTIMEDIA"). However, we recommend that if you're using one of the pre-defined profile names listed above, you should use the exact capitalization as shown.

In practical terms, the presentation profile is used by some interpreters to select a default set of visual settings to use when starting the game. For example, HTML TADS for Windows looks for a "theme" that has the same name as the presentation profile, and uses the matching theme, if any, when starting the game. An HTML TADS theme is simply a set of font, color, and other visual settings. Other interpreters, including all of the current text-only interpreters, completely ignore the presentation profile setting. Authors mustn't expect a presentation profile to select any particular color or font scheme or to have any other specific effects, because it's up to each interpreter to determine how to use the profile setting, if at all.

Note that the presentation profile does not have any effect at all on the capabilities of the interpreter: the profile setting doesn't turn off any features an interpreter would otherwise offer, and it doesn't limit what kind of interpreter can be used to play the game. Selecting the "plain text" profile, for example, does not disable graphics or sound in an interpreter; it simply gives the interpreter guidance that the author feels the game will look best when displayed in a style (fonts, colors, etc.) suitable for traditional text-only games. Similarly, selecting the "multimedia" profile doesn't prevent the game from being played on text-only interpreters; it merely hints to interpreters that they should use a visual style suited for a more diverse mixture of text effects and/or graphics.

5.10.3. `<glulx>`

This section contains only optional tags.

```
<glulx>
    <release>7</release>
    <serial>930723</serial>
    <checksum>FF3DABC6</checksum>
    <coverpicture>1</coverpicture>
    <presentationprofile>Plain Text</presentationprofile>
    <width>640</width>
    <height>480</height>
</glulx>
```

The purpose of these tags is to provide a portable replacement for the common practice of releasing a WinGlulx-specific configuration file to achieve the same effect.

5.10.3.1. `<coverpicture>`

See §5.10.1.1, "<coverpicture>".

5.10.3.2. `<presentationprofile>`

See §5.10.2.1, "<presentationprofile>".

5.10.3.3. `<width>`, `<height>`

The suggested dimensions of the game's display. `<width>` and `<height>` are optional, but if one is defined, the other must be as well. Interpreters are free to disregard this entirely. It is suggested that interpreters constrain their display size to reflect the aspect ratio indicated by these values – though they should not force the user to use these exact dimensions. These exact values may be used to set the initial size of the display if possible (though they should be scaled to not exceed the size of the user's screen).

5.10.4. <hugo>

This section contains only optional tags, and is reserved for later definition by Kent Tessman on behalf of Hugo.

5.10.5. <adrift>

This section contains only optional tags, and is reserved for later definition by Campbell Wild on behalf of ADRIFT.

5.11. Releases

The <releases> section is optional. It is important to stress that the bulk of an iFiction record – and in particular its <identification> and <bibliographic> sections – document a work of IF throughout all the editions it has, not any individual release of that work.

Information about specific releases may be placed in this section:

```
<releases>
    <attached>
        <release>
        ...
        </release>
    </attached>
    <history>
        <release>
        ...
        </release>
        <release>
        ...
        </release>
        ...
    </history>
</releases>
```

5.11.1. <attached> and <history>

The <attached> tag is optional. It may only exist in an iFiction record attached to a specific story file. So, for instance, a compiler of a story file is entitled to write about the <attached> release: but the IF-archive, when serving an iFiction record to the public based on an IFID alone, must not serve <attached> – it contains data which is not determined by the IFID.

<history> is optional, and can legally be served by the IF-archive. It consists of a release history for the work, and contains one or more <release> blocks, which must be all distinct from each other (i.e., no two <release> blocks in the <history> can contain identical data). <release> blocks may be placed in any order.

It is not required that the release in the <attached> tag, if there is one, be included in the <history>.

It is not required that any release in the <history> list should have the same date as the one given in <firstpublished>. In general, a <history> list is not obliged to be complete.

5.11.2. <release>

A <release> should have the form:

```
<release>
    <version>3</version>
    <releasedate>2005-11-30</releasedate>
    <compiler>Inform 7</compiler>
    <compilerversion>3D26</compilerversion>
</release>
```

where <releasedate> is compulsory but the other two tags are optional.

<releasedate> has the same format as <firstpublished>: YYYY or YYYY-MM-DD.

<version> must be a non-negative integer.

<compiler> is the design system used to compile the story file. Only very major version numbers should be present, if at all (e.g. "TADS 3" or "Inform 6" but not "Inform 6.12").

<compilerversion> may only be present if <compiler> is present. Different compilers will have different notations for their versions.

The release date can be the date of compilation of the released story file, if known, rather than the day on which commercial distribution began.

5.12. Colophon

The <colophon> tag is optional. It identifies the tool which wrote the .iFiction record, and also the date at which the data originates. Presented with multiple .iFiction records for the same story, a utility should use the colophon to determine which is the most recent.

The colophon may be updated whenever a program or editor makes substantive changes to the .iFiction record. Operations which do not change the content of the .iFiction record (such as extraction from, or aggregation to, a multi-record file, or encapsulation within a wrapper or story file) should not change the colophon.

Example:

```
<colophon>
  <generator>Inform 7</generator>
  <generatorversion>3F47</generatorversion>
  <originated>2006-04-12</originated>
</colophon>
```

5.12.1. <generator>

<generator> is mandatory within <colophon>.

This is the name of the system which created this iFiction record. For iFiction files generated by hand, this should be the name of the person or organization responsible for generating the iFiction file.

At this time, likely values for <generator> include:

```
Inform 7        - for metadata generated by the inform compiler
Zoom            - for metadata generated by the Zoom interpreter
Babel           - for metadata generated by babel*
ifarchive.org   - for metadata compiled by the maintainers of the if-
archive
```

* Though babel extracts iFiction files from various sources, it only generates a colophon when it is creating or modifying metadata, as in the case where it is synthesizing metadata from a story file which

does not contain an iFiction record as such (e.g., when synthesising an iFiction record from the GameInfo structure inside a TADS story file).

5.12.2. <generatorversion>

<generatorversion> is optional within <colophon>.

<version> refers to the version of the generator used to produce the file. This can be used to isolate cases where an obsolete tool has been used to produce metadata. Story file compilers should follow the same conventions here as for <compiler> and <compilerversion> in a <release> tag (see above).

Babel uses the revision of this treaty which it supports. Hand-written .iFiction files should not use <generatorversion>.

5.12.3. <originated>

<originated> is mandatory within <colophon>.

This is the best estimate of the date at which the bibliographic data in the iFiction record was last approved by the author of the work being described. This should be the date most useful in determining the freshness of the .iFiction data.

If a design system is compiling a story file and including an iFiction record with it, then <originated> will normally be the compilation date.

However, a third-party tool which synthesizes metadata from a story file should use either the compilation date (if it can be determined) of the story file, or else the compilation date of the tool itself. It should *not* use the date on which synthesis occurred, as this would result in obsolete metadata receiving current dates.

Abstractly, the <originated> tag should contain the earliest date on which this metadata could have been created, not necessarily the date on which it actually was created.

5.13. Annotation

The <annotation> tag is optional. For an iFiction record embedded in a story file or otherwise produced by a design system, it is forbidden. It will not normally be present in any iFiction record served by the IF-archive.

The purpose of the <annotation> section is to allow tools such as iTunes-like browsers to store user-specified or other convenient information, without violating the iFiction schema.

Any such tool should use a subsection under its own name.

5.13.1. <zoom>

Reserved for the use of Andrew Hunter's interpreter "Zoom". Its tags include:

```
<comment>
    Any user comments on the story
<rating>
    The user rating, a value between 0 and 10 where 0=worst and
    10=best.
<story>
    The URL of the story file
<graphics>
    The URL of a file containing graphics for the story
```

```
<sounds>
    The URL of a file containing sounds for the story
    (note that these are likely to be local file URLs, and may change
    as Zoom reorganises its collection; any two or all three may
    coincide, if a blorb is used)
<savegame tag="Description" date="savedate">
    The URL of a saved game for the story. (There can be several
    such tags.) The description text is shown to the player as one
    of the restore options, and Zoom creates this by summarising the
    status bar contents when the game is saved. The date specifies
    when the game was saved.
```

5.14. Examples

5.14.1. "Trinity"

```
<?xml version="1.0" encoding="UTF-8"?>
<ifindex version="1.0"
xmlns="http://babel.ifarchive.org/protocol/iFiction/">
    <!-- Bibliographic data contributed by Graham Nelson -->
    <story>
        <identification>
            <ifid>ZCODE-12-860926</ifid>
            <ifid>ZCODE-11-860509</ifid>
            <format>zcode</format>
        </identification>
        <bibliographic>
            <title>Trinity</title>
            <author>Brian Moriarty</author>
            <language>en-US</language>
            <headline>An Interactive Fantasy</headline>
            <firstpublished>1986</firstpublished>
            <genre>Fantasy</genre>
            <group>Infocom</group>
            <description>"The time is out of joint; O curse spite,
            That ever I was born to set it right!" - Hamlet I.v.
            <br/>
            It's the last day of your $599 London vacation.
Unfortunately,
            it's also the first day of World War III. Only seconds
remain
            before an H-bomb vaporizes the city... and you with it.
            <br/>
            Unless you escape to another time, another dimension.
            <br/>
            For every atomic explosion unlocks the door to a secret
            universe; a plane between fantasy and reality, filled with
            curious artifacts and governed by its own mischievous
logic.
            You'll crisscross time and space as you explore this
            fascinating universe, learning to control its inexorable
power.
            <br/>
            Trinity leads you on a journey back to the dawn of the
atomic
            age... and puts the course of history in your hands.
</description>
```

```
        </bibliographic>
        <cover>
            <format>jpg</format>
            <height>120</height>
            <width>120</width>
            <description>A sundial resting on sand, with an atomic
mushroom
            cloud in the background.</description>
        </cover>
        <contacts>
            <url>http://en.wikipedia.org/wiki/Infocom</url>
        </contacts>
        <releases>
            <history>
                <release>
                    <version>12</version>
                    <releasedate>1986-09-26</releasedate>
                    <compiler>ZILCH</compiler>
                    <compilerversion>4</compilerversion>
                </release>
                <release>
                    <version>11</version>
                    <releasedate>1986-05-09</releasedate>
                    <compiler>ZILCH</compiler>
                    <compilerversion>4</compilerversion>
                </release>
            </history>
        </releases>
    </story>
</ifindex>
```

5.14.2. "Bronze"

```
<?xml version="1.0" encoding="UTF-8"?>
<ifindex version="1.0"
xmlns="http://babel.ifarchive.org/protocol/iFiction/">
    <story>
        <identification>
            <ifid>1810847C-0DC7-44D5-94EF-313A3E7AF257</ifid>
            <format>zcode</format>
        </identification>
        <bibliographic>
            <title>Bronze</title>
            <headline>A fractured fairy tale</headline>
            <language>en-US</language>
            <author>Emily Short</author>
            <genre>Fairy Tale</genre>
            <description>When the seventh day comes and it is time for
you
            to return to the castle in the forest, your sisters cling
to your
            sleeves.<br/>'Don't go back,' they say, and 'When will we
ever see
            you again?' But you imagine they will find consolation
            somewhere.<br/>Your father hangs back, silent and moody.
He has
            spent the week as far from you as possible, working until
```

```
late at
            night. Now he speaks only to ask whether the Beast treated
you
            'properly.' Since he obviously has his own ideas about
what must
            have taken place over the past few years, you do not reply
beyond
            a shrug.<br/>You breathe more easily once you're back in
the
            forest, alone.<br/>Bronze is a puzzle-oriented adaptation
of
            Beauty and the Beast with an expansive geography for the
inveterate
            explorer.<br/>Features help for novice players, a detailed
adaptive
            hint system to assist players who get lost, and a number
of
            features to make navigating a large space more pleasant.
</description>
            <firstpublished>2006</firstpublished>
            <group>Inform</group>
        </bibliographic>
        <resources>
            <auxiliary>
                <leafname>Bronze Manual.pdf</leafname>
                <description>Manual</description>
            </auxiliary>
            <auxiliary>
                <leafname>map.pdf</leafname>
                <description>Complete (Spoilerful) Map</description>
            </auxiliary>
            <auxiliary>
                <leafname>solution.txt</leafname>
                <description>Walkthrough</description>
            </auxiliary>
        </resources>
        <cover>
            <format>jpg</format>
            <height>960</height>
            <width>960</width>
            <description>An antique Chinese bronze bell.</description>
        </cover>
        <zcode>
            <serial>060329</serial>
            <release>1</release>
            <compiler>Inform 7 build 3G08</compiler>
            <coverpicture>1</coverpicture>
        </zcode>
        <colophon>
            <generator>Inform 7</generator>
            <generatorversion>3G08</generatorversion>
            <originated>2006-03-29</originated>
        </colophon>
    </story>
</ifindex>
```

5.14.3. "Ditch Day Drifter"

```
<?xml version="1.0" encoding="UTF-8"?>
<ifindex version="1.0"
xmlns="http://babel.ifarchive.org/protocol/iFiction/">
    <!-- Bibliographic data from www.tads.org/howto/gameinfo.htm -->
    <story>
        <identification>
            <ifid>TADS-C15B8633FF25B25DB1E61DE870D19D68</ifid>
            <format>tads2</format>
        </identification>
        <bibliographic>
            <title>Ditch Day Drifter</title>
            <author>Michael J. Roberts</author>
            <language>en-US</language>
            <headline>An Interactive Fiction</headline>
            <firstpublished>1990-08-10</firstpublished>
            <genre>Collegiate Fiction</genre>
            <group>TADS</group>
            <description>You're an undergraduate at Caltech, where you
            wake up to find it's Ditch Day, the day when the seniors
ditch
            classes and leave "stacks" for the underclassmen to
            solve.<br/>The original TADS sample game.</description>
        </bibliographic>
        <tads2>
            <version>1.0</version>
            <releasedate>1990-08-10</releasedate>
            <htmldescription>You're an undergraduate at Caltech, where
you
            wake up to find it's Ditch Day, the day when the seniors
ditch
            classes and leave &ldquo;stacks&rdquo; for the
underclassmen
            to solve.<p><i>The original TADS sample game.</i>
</htmldescription>
            <presentationprofile>Default</presentationprofile>
        </tads2>
    </story>
</ifindex>
```

5.14.4. "Dungeon Adventure"

Level 9 games are an interesting test case, since although Level 9 did have a reasonably well-defined virtual machine, its story files were only ever circulated embedded in interpreters: a tool by Paul David Doherty cuts out the story file, but does fairly horrible things to achieve this. There is no good identification procedure to tell whether an arbitrary file is a Level 9 story file: and the IFID below is another case where we have to resort to an MD5 checksum.

```
<?xml version="1.0" encoding="UTF-8"?>
<ifindex version="1.0"
xmlns="http://babel.ifarchive.org/protocol/iFiction/">
    <!-- Bibliographic data contributed by Graham Nelson -->
    <story>
        <identification>
            <ifid>4C1C055EF01B5930744287EF5A0DCB14</ifid>
            <format>level9</format>
        </identification>
        <bibliographic>
```

```
            <title>Dungeon Adventure</title>
            <author>Pete Austin, Mike Austin and Nick Austin</author>
            <language>en-UK</language>
            <firstpublished>1984</firstpublished>
            <genre>Fantasy</genre>
            <group>Level 9</group>
            <series>Middle Earth Trilogy</series>
            <seriesnumber>3</seriesnumber>
            <description>Complete our Middle Earth Trilogy. The Demon
has
            been defeated and his Dark Tower cast down. But its
dangerous
            chambers remain, filled with hoarded treasure and magic.
There
            are just two snags. Other creatures want the loot, as
well, and
            many guardians remain: skeletons, carnivorous jellies,
black
            balls etc. Even an orc or two. Success will not come
            easily!</description>
        </bibliographic>
        <contacts>
            <url>http://www.if-
legends.org/~l9memorial/html/home.html</url>
        </contacts>
    </story>
</ifindex>
```

5.15. The sparse iFiction format

For legacy story files and for story files generated by a system which does not provide iFiction support, it may be necessary to write .iFiction data by hand. It would be tedious and error-prone to insert IFIDs by hand, but the Babel software is able to fill in missing IFIDs. An iFiction record which is missing its identification is called "sparse".

Formally, a "sparse" .iFiction record conforms to the same specification as a normal .iFiction file, except that the `<identification>` section is optional.

Babel provides a facility to "complete" a sparse .iFiction file, turning it into a complete .iFiction file by analyzing a story file.

This process has the following caveats:

- If no `<identification>` section is found in the sparse .iFiction, a new one will be added
- If an `<identification>` section is found, the `<format>` tag must match the format of the provided story file
- If the `<ifid>` of the provided story file is not found, it will be added.

(An `<identification>` section is allowed in sparse .iFiction to deal with legacy projects which have more than one IFID.)

A. Appendix: Babel: a user's guide

The specification below is *not* formally part of the treaty, and exact syntaxes may change, but it seems worth documenting the tool here, particularly since it will likely be used by all of the parties to the treaty.

A.1. Using babel on the command line

In the specification below, <storyfilename> must not be a zip archive (i.e. end in ".zip" with some casing), or a .iFiction file: babel should halt with an error if so.

Conversely, <ifictionfilename> must be a ".iFiction" file.

```
babel -ifid <storyfilename>
```

Examines the named story file and prints output as follows:

```
IFID: 1974A053-7DB0-4103-93A1-767C1382C0B7
```

Note that babel must determine the IFID using the algorithm in 2.2 above – in particular, falling back on MD5 if all else fails. It is therefore not possible for this usage to fail to print a IFID (so long as the file is not a zip archive, no file-system accidents occur, etc.). If more than one IFID is recorded, because the story file contains metadata which lists more than one IFID, a list is printed:

```
IFID: 731F0480-5CB5-4340-B65B-384FC6B1F5B4
IFID: ZCODE-8-040205-6630
```

```
babel -identify <storyfilename>
```

Prints out a brief summary of the story file, in the following form:

```
"When in Rome", by Emily Short
IFID: XXXXXXXXXXXXXXXXXXXX
blorbed zcode, 517K, cover 960x960 jpeg

No bibliographic data
IFID: XXXXXXXXXXXXXXXXXXXX
zcode, 180K, no cover
```

Printing is done in a Terminal-safe way: the text of the title and author's name might be any Unicode characters, but all characters outside the Unicode range 0x20 to 0x7E will be flattened to underscores.

```
babel -ifid <ifictionfilename>
```

Similar, but outputs the IFIDs recorded in <ifid> tags in the iFiction file supplied: there must be at least one in each of the <story> sections in that file. (It is legal for an iFiction file which is separate from a story file to record data on more than one story: this usage of babel should list every IFID from every story.)

```
babel -format <storyfilename>
```

Examines the named story file and prints output as in the following examples:

```
Format: zcode
Format: blorbed glulx
Format: tads3
Format: unknown
```

This should be determined using the recognition algorithm determined below. The format name will be one of those used in the iFiction <format> tag, or "unknown" if there is no indication what it is. The prefix "blorbed" indicates that the story file is embedded in a blorb archive. (The combination "blorbed unknown" is impossible, since the blorb format explicitly requires a statement of the story file's format.)

```
babel -ifiction <storyfilename> [-to <directory>]
```

Extracts a .iFiction file to a file named XXXXX.iFiction, where XXXXX is the primary (first-listed) IFID; to the current working directory by default, and otherwise to the given "-to" directory.

If successful, babel prints:

```
Extracted XXXXX.iFiction
```

(or whatever leafname it has). If there is no iFiction record, babel prints

```
No iFiction record for XXXXX
```

This is printed to stdout, not stderr: it isn't an error.

```
babel -cover <storyfilename> [-to <directory>]
```

Identical to babel -ifiction, but for the cover art, which should have the filename XXXXX.jpg or XXXXX.png. On success, prints

```
Extracted XXXXX.jpg (960x960)
```

(or .png, as appropriate); the dimensions given are width x height. On failure,

```
No cover art for XXXXX
```

```
babel -verify <ifictionfilename>
```

Verifies the .iFiction file for adherence to this standard, to the best of babel's abilities. If the file is apparently correct, prints

```
Verified <first-IFID-in-file>
```

If not, prints suitable errors to stderr.

```
babel -fish <storyfilename> [-to <directory>]
```

A sort of super-extractor, this fishes out both the cover art and the iFiction record – where present – printing output as in -ifiction and -cover above. *However*, in the event that the story file records multiple IFIDs, copies are created for each IFID present, not simply for the first. Thus, for instance:

```
$ babel -fish Savoir.zblorb
Extracted 731F0480-5CB5-4340-B65B-384FC6B1F5B4.iFiction
Extracted ZCODE-8-040205-6630.iFiction
Extracted 731F0480-5CB5-4340-B65B-384FC6B1F5B4.jpg (960x960)
Extracted ZCODE-8-040205-6630.jpg (960x960)
$
```

```
babel -fish <ifictionfilename> [-to <directory>]
```

Each individual `<story>` record in the iFiction file is written out, as an iFiction file in its own right, and named in the usual XXXXX.iFiction way. Moreover, if a `<story>` records more than one IFID, a copy is made for each of these IFIDs.

(Thus, for instance, a single iFiction file containing data on all of the Level 9 games could be expanded into individual records suitable for use by the IF-archive.)

```
babel -lint <ifictionfile>
```

As "-verify", but also checks for compliance with the stylistic guidelines of this document. If it does, it will print

```
<IFID> conforms to iFiction style guidelines
```

If there are nonfatal problems, it will list them and then print

```
<IFID> does not conform to guidelines
```

If there are errors, neither of the above messages will appear.

```
babel -complete <storyfile> <ifictionfile>
```

Completes the sparse ifictionfile using information from storyfile. Writes the output to XXXX.iFiction, where XXXX is the ifid of the story.

```
babel -story <storyfile>
```

If the story file is inside a container format, extracts just the story file. If not, copies the input file. In either case, it creates XXXX.EXT, where XXXX is the ifid of the file, and ext is the recommended extension for the story file.

```
babel -unblorb <storyfile>
```

As "-fish", but also extracts the story file.

```
babel -blorb <storyfile> <ifictionfile> [<cover art>]
```

Creates a blorb container consisting of the story file, ifiction file and, optionally, the cover art. The resulting file is named XXXX.blorb where XXX is the ifid of the story file. If the ifiction file is sparse, it will be completed first.

A.1.1. Using babel in a pipe

As one of the intended uses of babel is as a back-end to other applications, it may be desirable to invoke the program as part of a pipeline. The following features are available to facilitate this, though the exact syntax of their use may vary according to the conventions of the host platform.

A.1.1.1. Piping data into babel

With the exception of -blorb, any of the operations listed above as taking `<ifictionfile>` as input can use the filename "`-`" to read from standard input instead. In UNIX-like syntax, the following two commands are equivalent:

```
babel -verify - < ZCODE-67-000000.iFiction
babel -verify ZCODE-67-000000.iFiction
```

This can be useful when coupled with a tool such as babel-get. For instance,

```
babel-get -ifiction ZCODE-67-000000 -url http://babel.ifarchive.org
    | babel -verify -
```

will download and verify the .iFiction record for Sorcerer.

A.1.1.2. Piping data out of babel

```
babel -meta <storyfile>
```

Extracts the iFiction record from the specified story file and prints it to standard output, so that it can be piped into a program which reads .iFiction. For example:

```
babel -meta lakeside.zblorb | babel -verify -
```

will verify the .iFiction data contained within lakeside.zblorb.

A.2. The Babel handler API

As a command-line utility, the babel tool performs basic analysis on both story files and iFiction records. The latter task is no more than routine XML parsing, but babel's functions to handle story files may be very helpful to tools such as interpreters and browsers, and it makes these functions available as the "babel handler", contained in the file babel_handler.c.

The babel handler performs the following tasks:

1. loads a story file from disk;
2. determines the format of the story file and selects the correct treaty functions for dealing with it;
3. seamlessly handles integration of container formats.

That is, once the babel handler has been initialized with a story file, it will forward treaty requests to the proper handler or to the container format's handler.

This is the babel_handler API:

```
void *get_babel_ctx(void);
void release_babel_ctx(void *);
```

These functions create and destroy an opaque context object used by the babel handler. Each context object describes one currently loaded file. If you wish to process files in parallel (as a multithreaded application might), you should use a separate context object for each file (that is, each thread. On the other hand, if you want to have separate threads handle, say, the cover art and metadata on the *same* file, they should use the same context object).

The object returned by `get_babel_ctx` is passed as the last parameter to each of the remaining functions:

```
char *babel_init_ctx(char *filename, void *);
```

Initializes the babel handler with the given file. This function must be called successfully before any other babel handler function can be called. On a successful load, this returns the name of the format. If this fails, it will return NULL. If it returns NULL, you must not use any of the other functions, except `babel_release_ctx`, `babel_md5_ifid_ctx` and `babel_get_length_ctx`.

```
char *babel_init_raw_ctx(void *story_file, int32 extent, void *)
```

Does the same thing, but uses a story file which has already been loaded into memory. (Note that this will preclude the babel_handler's usual file extension checking.)

```
int32 babel_treaty_ctx(int32 selector, void *output,
                       int32 output_extent, void *);
```

Calls the proper treaty module for the given selector (the story file and story file extent are ommitted, as these are inferred from the babel handler context). This function will correctly and invisibly handle container formats.

```
void babel_release_ctx(void *);
```

This releases the resources held by the given babel handler context. You must call this when you are done with the file loaded by `babel_init_ctx`, even if `babel_init_ctx` returned NULL. Once this is called, the babel handler context must not be used until it is re-initialized.

```
char *babel_get_format_ctx(void *);
```

This returns the format of the loaded file. For story files not inside a container, this is identical to the value returned by the `GET_FORMAT_NAME_SEL` treaty selector. For story files contained within a container format, the returned name is a composite, such as "blorbed zcode".

```
int32 babel_md5_ifid_ctx(char *buffer, int extent, void *);
```

This generates an IFID for the loaded story file using the last-resort method of using an MD5 hash. Note that if `GET_STORY_FILE_IFID_SEL` returns a positive number, this is not the actual IFID. This function can be used even if `babel_init_ctx` returned NULL. Returns nonzero so long as an MD5 can be generated (It will fail only if the file does not exist at all or the provided output buffer is less than 33 bytes long)

```
int32 babel_get_length_ctx(void *);
```

This function returns the length of the loaded story file in bytes.

```
int32 babel_get_story_length_ctx(void *);
```

As `babel_get_length_ctx`, but if the story file is in a container, returns just the length of the story itself.

```
int32 babel_get_authoritative_ctx(void *)
```

Returns true if the loaded story file's format has been positively identified.

```
void *babel_get_file_ctx(void *);
```

Returns a pointer to the actual loaded story file.

```
void *babel_get_story_file_ctx(void *);
```

As `babel_get_file_ctx`, but, if the story file is inside a container, return just the story file, not the container.

Applications which only need to deal with one story file at a time can use the non-parallelizable form of these functions:

```
char *babel_init(char *filename);
char *babel_init_raw(void *story_file, int32 extent);
int32 babel_treaty(int32 selector, void *output, int32 output_extent);
void babel_release(void);
char *babel_get_format(void);
int32 babel_md5_ifid(char *buffer, int32 extent);
int32 babel_get_length(void);
int32 babel_get_file(void);
int32 babel_get_authoritative(void);
```

These do not take a context object, instead using a "default" context.

babel_handler.c relies on the module registry, contained in register.c and modules.h. modules.h contains macros which announce the known story and container formats.

It also relies on misc.c for the `my_malloc` function. You may provide your own replacement for `my_malloc` if you use some other allocator.

A.3. Support for general containers

For the purposes of babel, a container format is treated similarly to a story file format.

When contributing a new container format to babel, a container format should define the following function:

```
int32 CONTAINER_treaty(void *container_file, int32 container_extent,
                       void *output, int32 output_extent)
```

(`CONTAINER` should be replaced by the common name of the container format.) This function should perform the same functions as a `SYSTEM_treaty` function, except that it should accept three additional selectors:

	container_file	output
CONTAINER_GET_STORY_FORMAT_SEL	not null	not null
CONTAINER_GET_STORY_EXTENT_SEL	not null	null
CONTAINER_GET_STORY_SEL	not null	not null

`CONTAINER_GET_STORY_FORMAT_SEL` copies the treaty name of the format of the contained story file into the output buffer, returning >0 if all went well. The value output by this selector should be the same as would be output by calling `GET_FORMAT_NAME_SEL` on the corresponding treaty module. (Though this is no guarantee that `CLAIM_STORY_FILE_SEL` will accept the story file: `CONTAINER_GET_STORY_FORMAT_SEL` is not expected to do any more than consult the container's formatting data. For example, an erroneous blorb file may claim to contain a zcode story file when it actually contains a glulx.)

`CONTAINER_GET_STORY_EXTENT_SEL` returns the size, in bytes, of the story file contained in the container.

`CONTAINER_GET_STORY_SEL` copies the story file from the container into the output buffer, returning the number of bytes copied if all went well.

Because a container module is compatible with a story file treaty module, a simple tool may not need to distinguish between the two. However, any tool which needs to do sophisticated analysis should consult the `CONTAINER_treaty` function, then the `SYSTEM_treaty` function for the story file contained therein should the container not be able to ascertain the necesary information (especially for `GET_STORY_FILE_IFID_SEL`).

(Note: `GET_FORMAT_NAME_SEL` on a container format should return the name of the container format. Depending on your desired usage, you may want to check one, the other, or both. For instance, the babel tool composes both in the form "CONTAINERed SYSTEM": e.g., "blorbed zcode".)

A.4. Definitions made in treaty.h

treaty.h is a portable C header file which defines the various symbolic constants which should be used in treaty modules.

Not everything in treaty.h is required by the Treaty of Babel; it also includes several definitions to simplify the lives of programmers.

It defines these values:

Return values for the SYSTEM_treaty function:

```
NO_REPLY_RV
INVALID_STORY_FILE_RV
UNAVAILABLE_RV
INVALID_USAGE_RV
VALID_STORY_FILE_RV
INCOMPLETE_REPLY_RV
```

Cover art types:

```
PNG_COVER_FORMAT
JPEG_COVER_FORMAT
```

SYSTEM_treaty selectors:

```
GET_HOME_PAGE_SEL
GET_FORMAT_NAME_SEL
GET_FILE_EXTENSIONS_SEL
CLAIM_STORY_FILE_SEL
GET_STORY_FILE_METADATA_EXTENT_SEL
GET_STORY_FILE_COVER_EXTENT_SEL
GET_STORY_FILE_COVER_FORMAT_SEL
GET_STORY_FILE_IFID_SEL
GET_STORY_FILE_METADATA_SEL
GET_STORY_FILE_COVER_SEL
```

These bitmasks are provided to help group selectors by their requirements:

```
TREATY_SELECTOR_INPUT           if the selector requires a story file
TREATY_SELECTOR_OUTPUT          if the selector requires an output
buffer
TREATY_SELECTOR_NUMBER          removes these flags from the selector
```

The treaty insists that programs supply output buffers at least this big:

```
TREATY_MINIMUM_EXTENT           currently 512 bytes
```

treaty.h also reserves three selectors for container formats:

```
CONTAINER_GET_STORY_FORMAT_SEL
CONTAINER_GET_STORY_EXTENT_SEL
CONTAINER_GET_STORY_FILE_SEL
```

and TREATY_CONTAINER_SELECTOR, which is a bitmask to detect these three selectors.

treaty.h also declares two types:

```
TREATY          a pointer to a SYSTEM_treaty function
int32           a 32-bit signed integer
```

A.5. The Babel iFiction API

The Babel iFiction API is a simple XML parser geared toward the metadata format used by this treaty. It is no substitute for a proper XML parser, but will suffice for simple programs which don't wish to be encumbered by a heavy-weight XML library.

The Babel iFiction API is made up from the files ifiction.c, and register_ifiction.c. It also uses the `my_malloc` function found in misc.c.

The iFiction API consists of these methods:

```
int32 ifiction_get_IFID(char *metadata, char *output, int32
output_extent);
```

Copies a comma separated list of the IFIDs found in the metadata into output. Returns the number of IFIDs found. Note that this may be 0 if the metadata is a sparse iFiction record: if the record is known not to be sparse, a return value of 0 indicates there is something wrong with the metadata.

```
char *ifiction_get_tag(char *md, char *p, char *t, char *from);
```

Returns the content of tag t within container tag p (so ifiction_get_tag(md, "bibliographic", "title") would return the title of a work), or NULL if it is not found. This function returns newly allocated heap memory, which must be freed by the calling context. To find the first matching tag in the file, pass the value NULL as the final parameter. To find subsequent occurences, pass the value returned by the previous call.

```
void ifiction_parse(char *md, IFCloseTag close_tag, void *close_ctx,
                    IFErrorHandler error_handler, void *error_ctx);
```

Processes md. `close_tag(tag,close_ctx)` is called as each tag is closed. `error_handler(char *, error_ctx)` is passed a string for any error encountered. You may print errors or ignore them, depending on your needs; processing will continue unless an error is very serious. (Note in particular that reading the "identification" section in a sparse iFiction record, which lacks such a section, will generate errors.)

The error_handler argument to ifiction_parse has the following signature:

```
void (*IFErrorHandler)(char *error_text, void *error_context);
```

The close_tag argument has this syntax:

```
void (*IFCloseTag)(struct XMLTag *tag, void *context);
```

Where struct XMLTag is defined as:

```
struct XMLTag
{
 int32 beginl;                  /* Beginning line number */
 char tag[256];                 /* name of the tag */
 char fulltag[256];             /* Full text of the opening tag */
 char *begin;                   /* Points to the beginning of the
tag's content */
 char *end;                     /* Points to the end of the tag's
content.
                                   setting *end=0 will turn begin into
a string
                                   containing the tag's content (But
if you do this, you
                                   should restore the original value
of *end before
                                   allowing control to return to the
ifiction parser) */
 char occurences[256];          /* reserved for internal use */
```

```
    char rocurrences[256];
    struct XMLTag *next;           /* The tag's parent */

};
```

A.6. The babel-get program

babel-get is a utility to retrieve metadata for a story. It requires that "curl" (https://curl.se/) be installed.

Usage:

```
babel-get -ifiction [<IFID>] -story <storyfile>
```

Prints the metadata for the given story. No output if metadata is not found or if an IFID which does not match the story file is given.

```
babel-get -ifiction [<IFID>] -ifiction <ifictionfile>
```

If an IFID is given, finds the corresponding ifiction record from the given file. If not, prints the first ifiction record in the given file.

```
babel-get -ifiction <IFID> -dir <ifiction directory>
```

Scans all ifiction files in the given directory for the ifiction record of the corresponding IFID and prints it.

```
babel-get -ifiction <IFID> -url <url>
```

Retrieves the ifiction record for the given IFID from the given server and prints it.

```
babel-get -cover [<IFID>] -story <storyfile> [-to <directory>]
```

Creates `<IFID>.jpg` or `<IFID>.png`, containing the cover art for the specified file.

```
babel-get -cover <IFID> -url <url> [-to <directory>]
```

Downloads `<IFID>.jpg` from the given server.

`<url>` should be an http://, https://, ftp:// or file:// style URL.